

第九章 设备与IO管理

提纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

提 纲

9.1设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.1 设备管理的功能和目标



磁带机



硬盘



光盘



绘图仪



扫描仪



键盘



打印机



显示器

设备管理的功能

- 设备管理的目标
 - 对设备资源进行合理的分配和调度，使用户在使用设备时不发生冲突并提高效率
 - 负责设备驱动和中断处理，使用户从繁琐的输入输出操作中解脱出来
- 现代设备管理采用缓冲和并行技术
 - 处理器速度比设备速度快数个量级
 - 设备与CPU之间，设备与设备之间并行
- 现代设备管理采用虚拟设备
 - 实设备数量较少，速度较慢
 - 利用共享设备来构造数量较多、速度较快的虚拟设备

设备管理的功能

- 设备分配与去配
 - 设备分配
 - 按照设备类型和相应分配算法决定将设备分配到哪一个要求该类设备的进程
 - 设备去配
 - 是指将设备资源由某占有资源的进程处收回
- 设备处理
 - 即设备驱动，实现 I/O 进程与设备控制器之间的通信
- 缓冲管理
 - 提高 CPU 和 I/O 设备之间操作的并行程度，尽量减少中断的次数

设备管理的目标

- 方便性
 - 按照用户的要求来控制I/O设备的工作，完成用户所希望的I/O操作，以减轻用户编制程序的负担
- 并行性
 - 通道技术和中断技术的引进，使外部设备和处理器的并行工作能力大大加强了
 - 但是，提供了并行性并不等于充分发挥了这种并行性。因此，操作系统的设备管理任务就是要充分发挥这种并行能力
- 均衡性
 - 多个进程竞争使用设备时，按一定策略分配和管理各种设备，使系统能有条不紊的工作
- 独立性
 - 用户在编制程序时，使用逻辑设备名，由系统实现从逻辑设备到物理设备（实际设备）的转换
 - 用户能独立于具体物理设备而方便使用设备

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.2 设备的分类

■ 按交互对象分

- 人机交互设备：显示设备、键盘、鼠标、打印机
- 与CPU等交互的设备：磁盘、磁带、传感器、控制器
- 计算机间的通信设备：网卡、调制解调器

■ 按交互方向分类

- 输入设备：键盘、扫描仪
- 输出设备：显示设备、打印机
- 双向设备：输入/输出：硬盘、软盘、网卡。

■ 按外设特性分类

- 使用特征：存储、输入/输出
- 数据传输率：低速（鼠标、键盘）、中速（打印机）、高速（网卡、磁盘、光盘）
- 信息组织特征：字符设备（打印机）、块设备（磁盘）、网络设备

9.2 设备的分类（Cont.）

- 按用途分类
 - 存储型设备
 - 磁盘，磁带，光盘，U盘
 - IO型设备
 - 输入设备：如mouse、keyboard、扫描仪等
 - 输出设备：如monitor 、打印机等
 - 网络设备
 - 网卡，交换机， etc.

9.2 设备的分类（Cont.）

- 数据传输的基本单位
 - 块型设备
 - 磁盘，磁带，光盘
 - 字符型设备设备
 - 输入输出设备和通信设备
 - 时钟和显示器

9.2 设备的分类（Cont.）

- 从资源管理角度
 - 共享型设备（块型）
 - 信息交换以块为单位，多个进程的I/O操作以块为单位可以交叉
 - 使用这类设备时，无需申请或释放设备
 - 包括除磁带机之外的所有块型设备
 - 属于有结构设备，传输速度快，可寻址（即可随机读/写任一快）
 - 独占型设备（块型）
 - 信息交换以块为单位，多个进程的I/O操作以块为单位不能交叉
 - 磁带机，只能顺序访问
 - 独占型设备（字符型）
 - 信息交换以字符为单位，多个进程的I/O操作以字符为单位不能交叉
 - 使用时，向系统申请。时常采用中断I/O方式
 - 属于无结构设备，传输速率低，不可寻址

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.3 设备的物理特性

- 字符型设备的物理特性
 - 以**字符**为I/O传输的基本单位，每传送一个字符发生一次I/O中断。如打印机、键盘、鼠标等
 - 基本特征是**传输速率低、不可寻址**，在输入/输出时常采用中断驱动方式
 - 字符型设备属于独占型设备，为此需要提供打开和关闭操作，实现互斥
- 存储型设备的物理特性
 - 以**块**为单位进行数据传送，如磁盘。每传输完一块，发生一次中断
 - 基本特征是传输速率高、可寻址，磁盘设备的I/O常采用DAM方式
 - 块设备接口将磁盘的所有扇区从0到n-1依次编码，这样将三维结构变成一种线性序列

磁带的物理特性

磁带的物理特性：启停设备 间隙



规格：1in, 16磁道；0.5in, 9磁道(现可达18磁道)。 每磁道1个磁头。

操作：反绕至带头，正向查找，反向查找，读，写，...

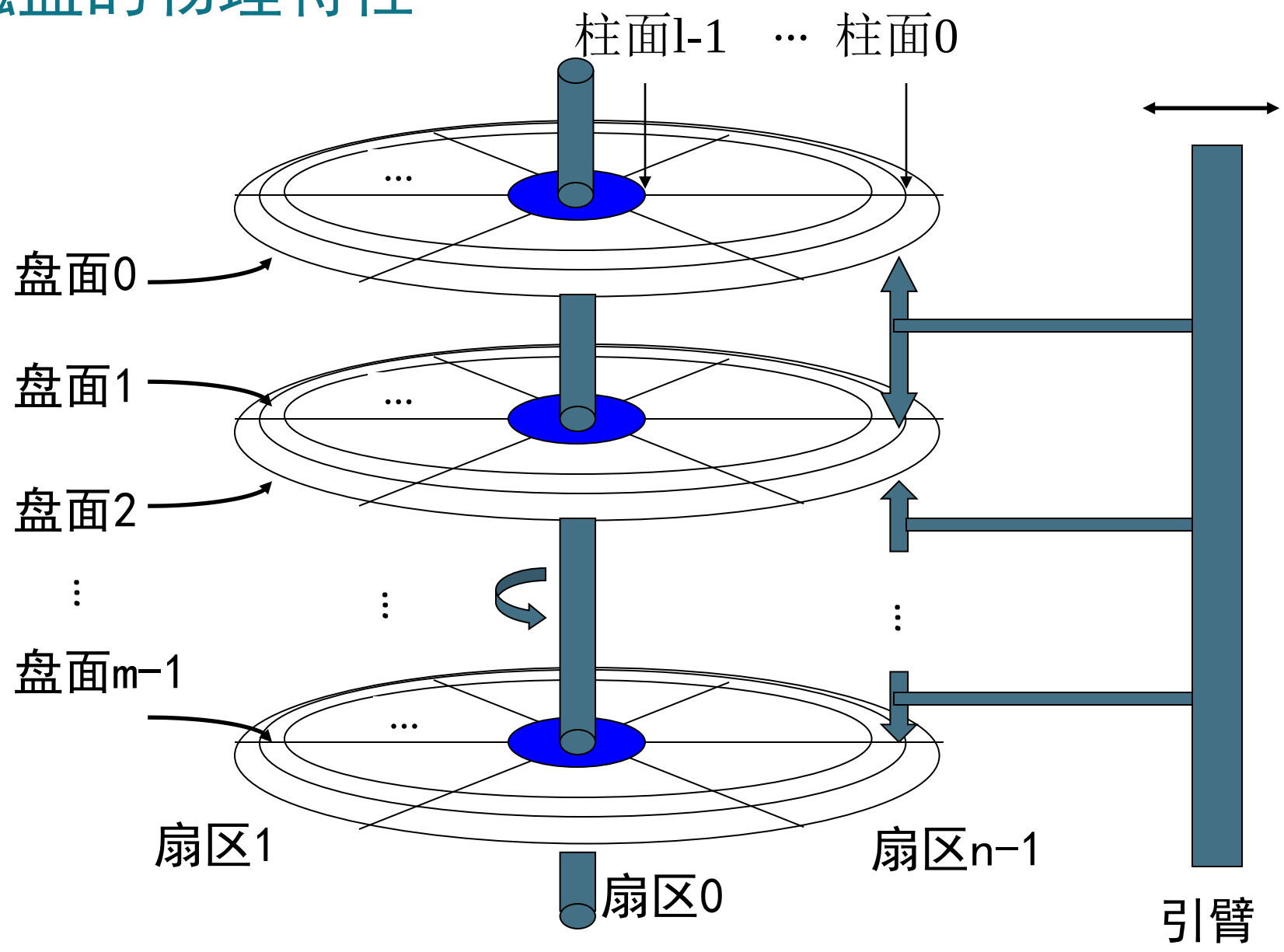
地址：一维

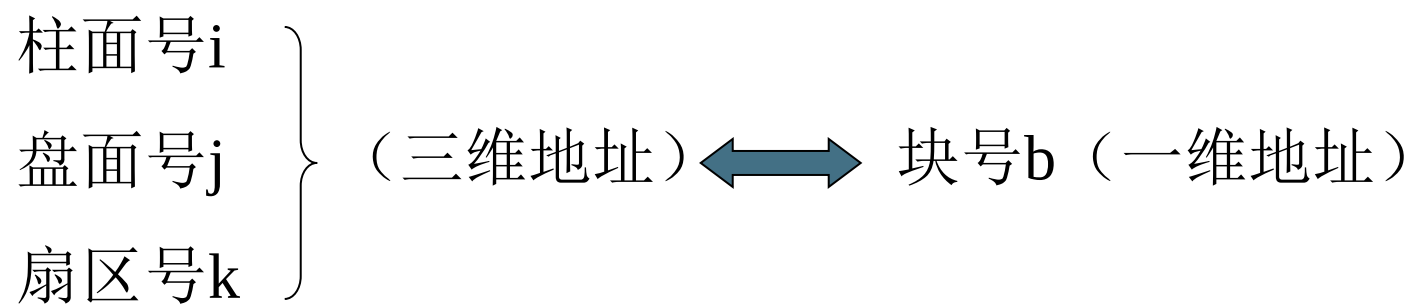
文件：顺序结构（一个文件占若干连续块）

磁盘的物理特性



磁盘的物理特性





编址方法：使相邻块物理上最近

例子：l=2; m=3; n=3

柱面号：	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
盘面号：	0	0	0	1	1	1	2	2	2	0	0	0	1	1	1	2	2	2
扇区号：	0	1	2	0	1	2	0	1	2	0	1	2	0	1	2	0	1	2
块 号：	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

三维地址 $\Rightarrow$ 一维地址：

$$b = i \times m \times n + j \times n + k$$

一维地址 $\Rightarrow$ 三维地址：

$$i = b \div (m \times n)$$

$$j = b \bmod (m \times n) \div n$$

$$k = b \bmod (m \times n) \bmod n$$

i ：柱面数

m ：盘面数

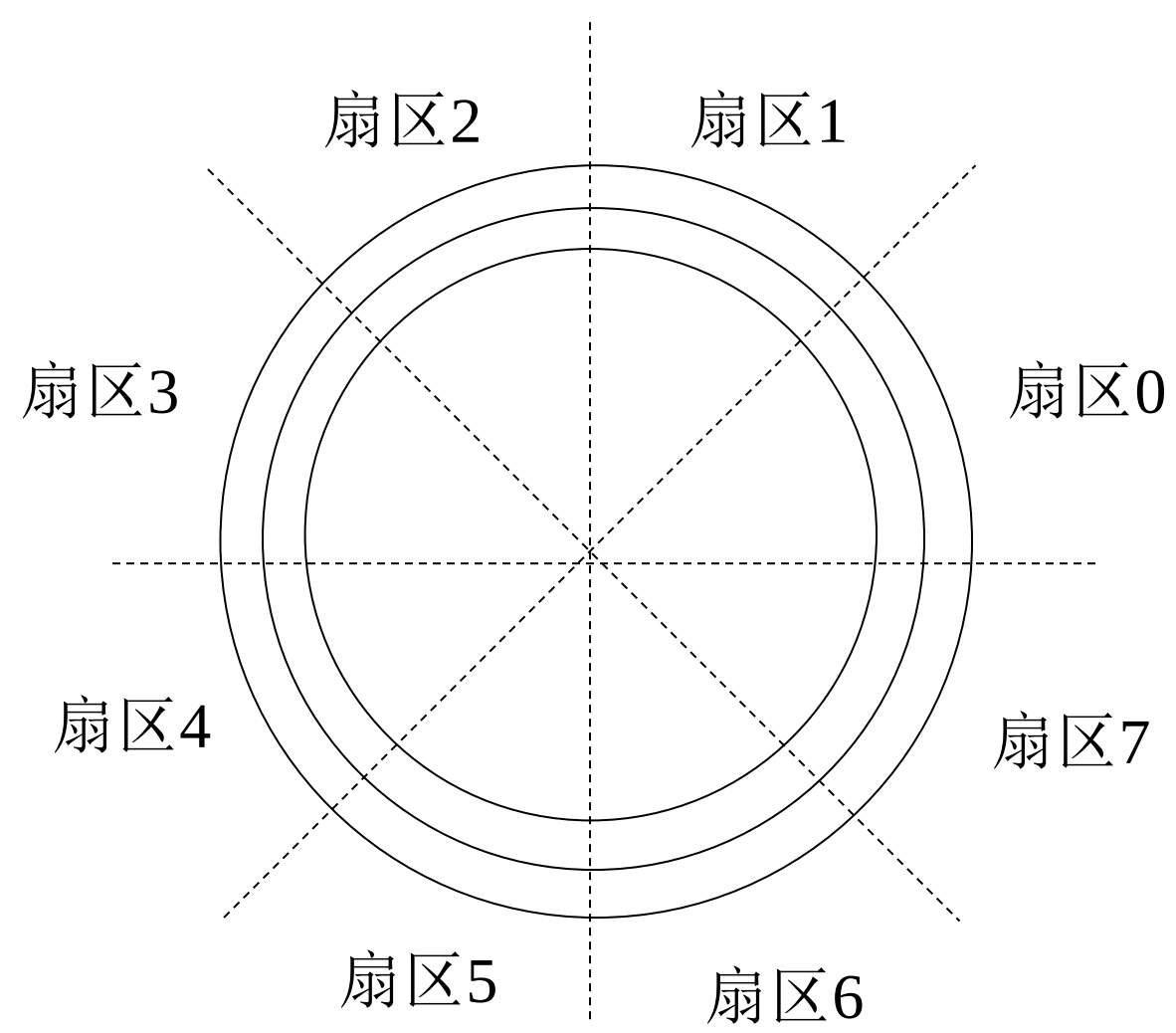
n ：扇区数

i ：柱面号

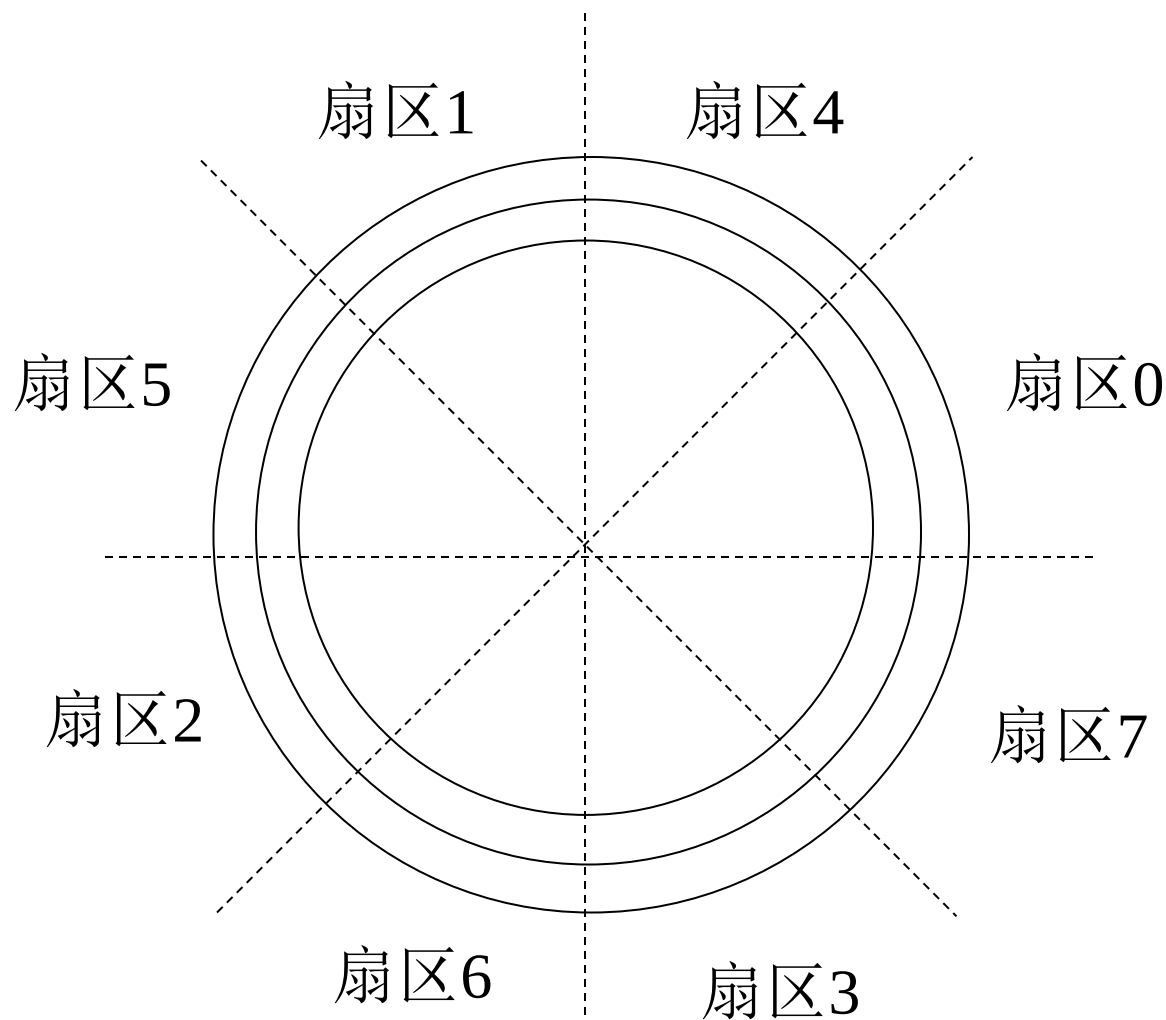
j ：盘面号

k ：扇区号

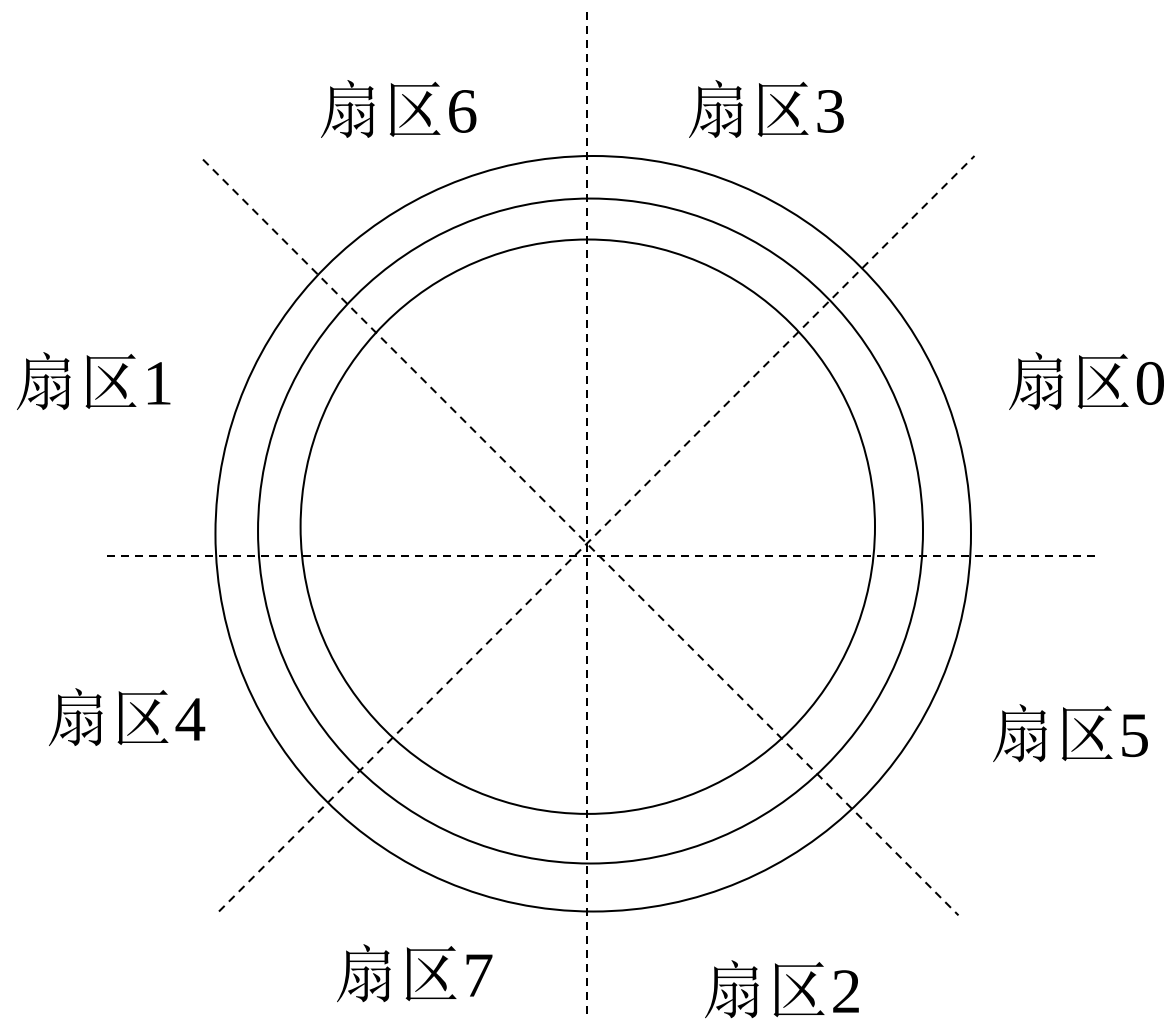
未考虑读写延迟的扇区编号：



考虑读写延迟的扇区编号(单交错)：



考虑读写延迟的扇区编号(双交错)：



提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.4 I/O 传输方式

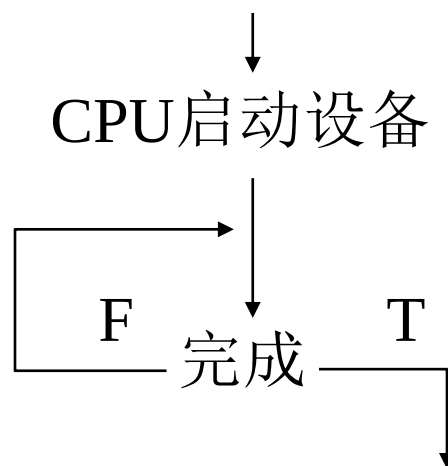
- IO操作演变历史
 - 程序查询方式(programmed IO) (polling)
 - CPU and Device can not work in parallel
 - 中断方式(interrupt)
 - CPU and device can work in parallel, too many interrupts for CPU
 - 直接内存方式(DMA)
 - DMA controller in charge of block io
 - 通道方式(channel)
 - special processor for dealing with io operations

9.4.1 程序控制查询方式

- 探询(polling) 又称程序控制输入输出，是最早的输入输出控制方式
 - 可以采用硬件提供的专用的输入输出指令，
 - 可采用内存操作指令完成
 - 将设备地址映射为内存地址空间的一部分即内存映射输入输出
 - 是硬件通常提供的输入输出指令形式
- 处理器代表进程向相应的设备模块发出输入输出请求，然后处理器反复查询设备的状态，直至输入输出完成。

9.4.1 程序控制查询方式

Pooling:



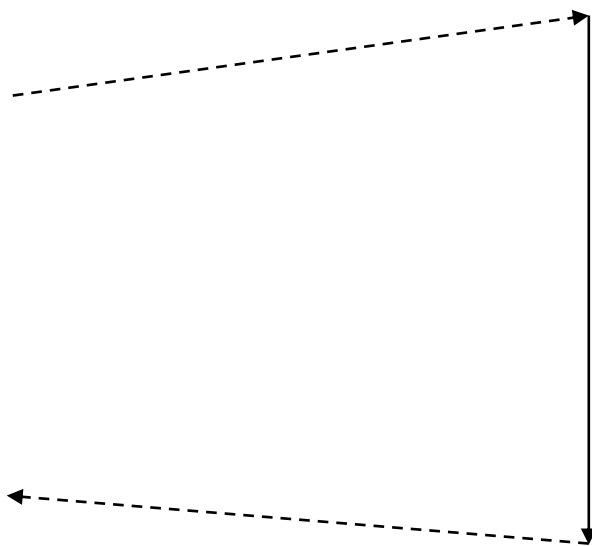
9.4.2 中断驱动方式

■ CPU

- 计算
- 启动设备
- 计算
- ...
- 计算
- 中断处理
- 计算

设备：

工作



9.4.3 内存映射方式

- 对应显示器等I/O传输不采用IN和OUT指令，而是采用取数和送数指令（MOV）
- 操作简单具有即时完成的特点，即指令执行完输入输出操作也就执行完了，没有中断需要处理
- `MOV 6, R0`
- `MOV R0, 6`

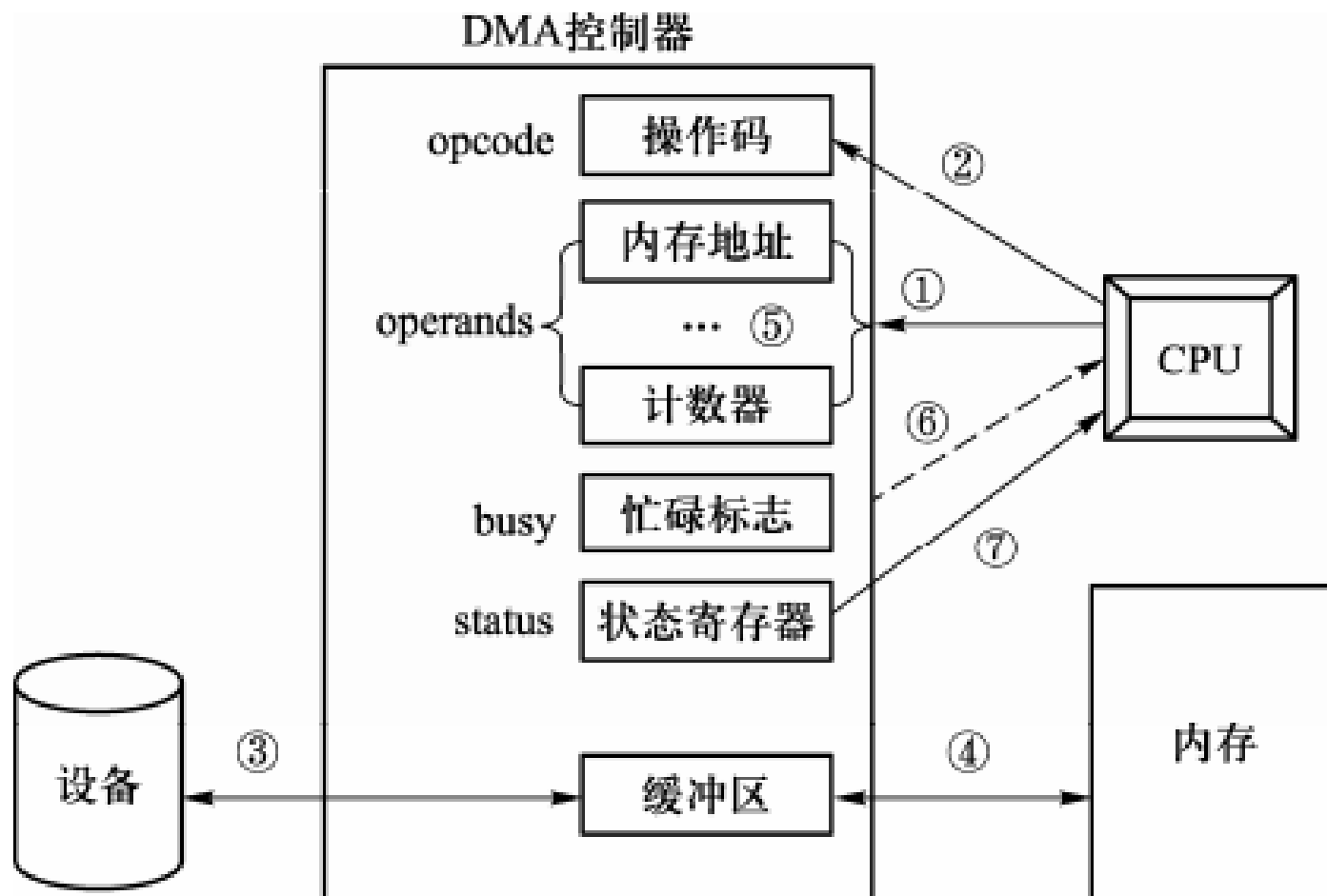
9.4.4 DMA方式

- DMA方式
 - 一种不经过CPU而直接从主存存取数据的数据交换模式
 - 通过总线在外设和主存之间开辟直接的数据交换通路
- DMA方式的特点
 - 数据传送的基本单位是数据块
 - 所传送的数据是从设备送内存，或者相反
 - 硬件要提供 DMA 控制器(有时与设备控制器集成在一起)
 - 整块数据的传送是在DMA控制器的控制下完成的，每传送一块，向CPU发送一次中断

9.4.4 DMA方式（Cont.）

- 在开始DMA传输时，主机向内存写入**DMA命令块**，向DMA控制器写入该命令块的地址，启动I/O设备
- 然后，CPU继续执行其它工作，DMA控制器则继续直接操作**内存总线**，将地址放到总线上开始传输。整个传输完成后，DMA控制器中断CPU

9.4.4 DMA方式 (Cont.)



9.4.4 DMA方式（Cont.）

- ①当CPU收到I/O设备的DMA请求时，CPU将操作量送到DMA控制器的操作数寄存器operands中，包括内存起始地址、传输量等；
- ②CPU将操作码送到DMA控制器的opcode寄存器以启动DMA控制器。DMA控制器将其忙碌寄存器置位，表明在此期间不再接受新的操作命令。此外，CPU可以执行与该控制器无关的其它操作；
- ③DMA控制器与设备交往，将数据由缓冲区传送到设备或者由设备传送到缓冲区；
- ④DMA控制器将缓冲区内容复制到内存空间，或由内存空间复制到缓冲区（每次传输一个数据块，这个过程不需要CPU参与）；
- ⑤计数器值减1。若非0（未传送完）则转步骤③继续传输；否则转⑥；
- ⑥传输结束时，DMA控制器复位其忙碌寄存器，并向CPU发送中断请求；
- ⑦CPU读入并检测其DMA状态寄存器，以确认操作是否成功。

9.4.4 DMA方式（Cont.）

- DMA方式&中断驱动方式
 - 中断驱动方式在**每个数据（字节）**传送完成后中断CPU
 - DMA方式则是在所要求传送的**一个数据块**全部传送结束时中断CPU
 - 中断驱动方式的数据传送是在中断处理时由**CPU控制**完成
 - DMA方式的数据传送是在**DMA控制器**的控制下完成。不过，在DMA方式中，数据传输的方向、存放数据的内存始址及传送数据的长度仍由CPU控制

9.4.5 通道方式

- 在通道控制方式中，数据传送方向、存放数据的内存始址及传送的数据块长度均由一个专门负责输入/输出的硬件——通道来控制
- 通道是负责IO操作的处理机，独立于CPU，有自己的指令系统
- 通道方式是DMA方式的发展，可以进一步减少CPU的干预，即把一个数据块的读/写为单位的干预，减少为一组数据块的读/写及有关控制和管理为单位的干预
- 通道方式可以实现CPU、通道和I/O设备三者的并行操作，从而更有效的提高系统的资源利用率

9.4.5 通道方式

■ 通道

■ 负责I/O操作的处理机

■ 指令系统

- 基本操作：控制、读、写、转移、结束
- 指令格式：（操作码，传输量，特征位，地址）

■ 运控部件

- CAW：记录下一条通道指令存放的地址
- CCW：保存正在执行的通道指令
- CSW：记载通道、控制器、设备的状态
- CDW：暂存内存与设备之间I/O传输的数据
- 存储区域（与CPU共用内存，通道内有缓冲区）
 - 保存通道程序（通道指令的有序序列）；保存I/O数据

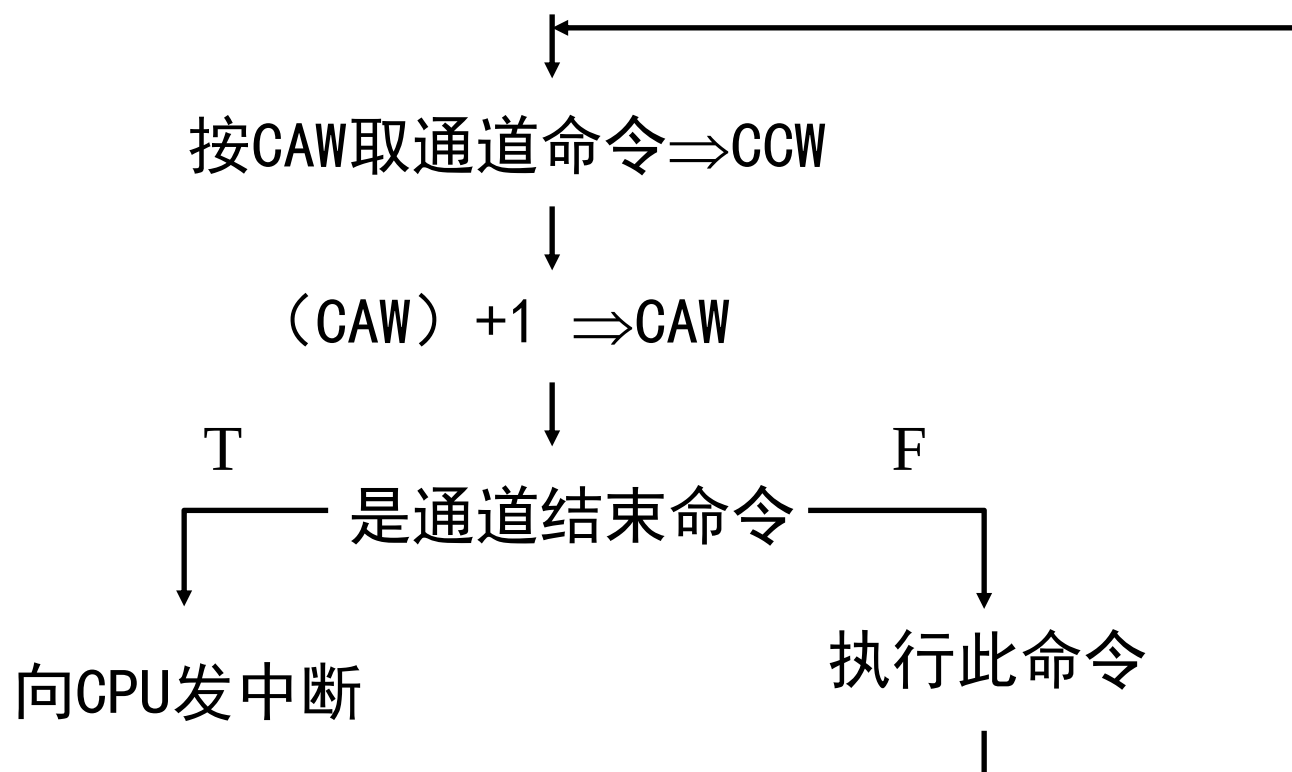
通道在I/O设备的作用

- 通道指令的类型单一，没有自己的内存
- 通道所执行的通道程序是放在主机的内存中，通道与CPU共享内存
- 一条通道指令可以传送一组数据，一个通道程序可以传送多组数据
- 多组数据全部传输完毕后才向处理器发一次中断，因而大大地减轻了主机的负担

通道方式

- 具有通道结构的计算机系统，主机、通道、控制器和设备之间采用四级连接，实施三级控制
 - 一个CPU连接若干通道；一个通道连接若干控制器；一个控制器连接若干设备
 - CPU通过执行I/O指令对通道实施控制，通道执行通道指令对控制器实施控制，控制器发出动作序列对设备实施控制，设备执行相应输入输出操作

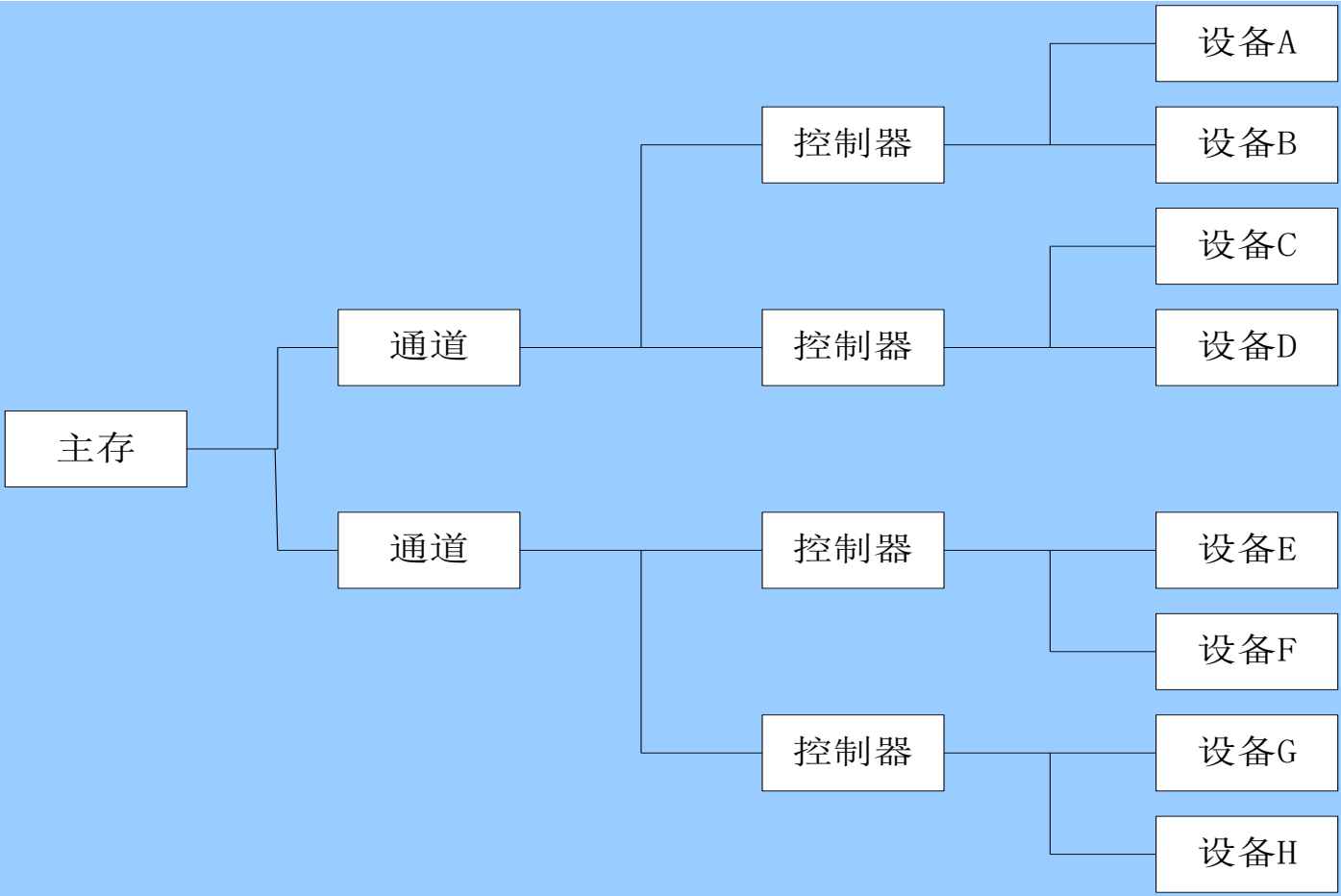
通道程序执行过程：



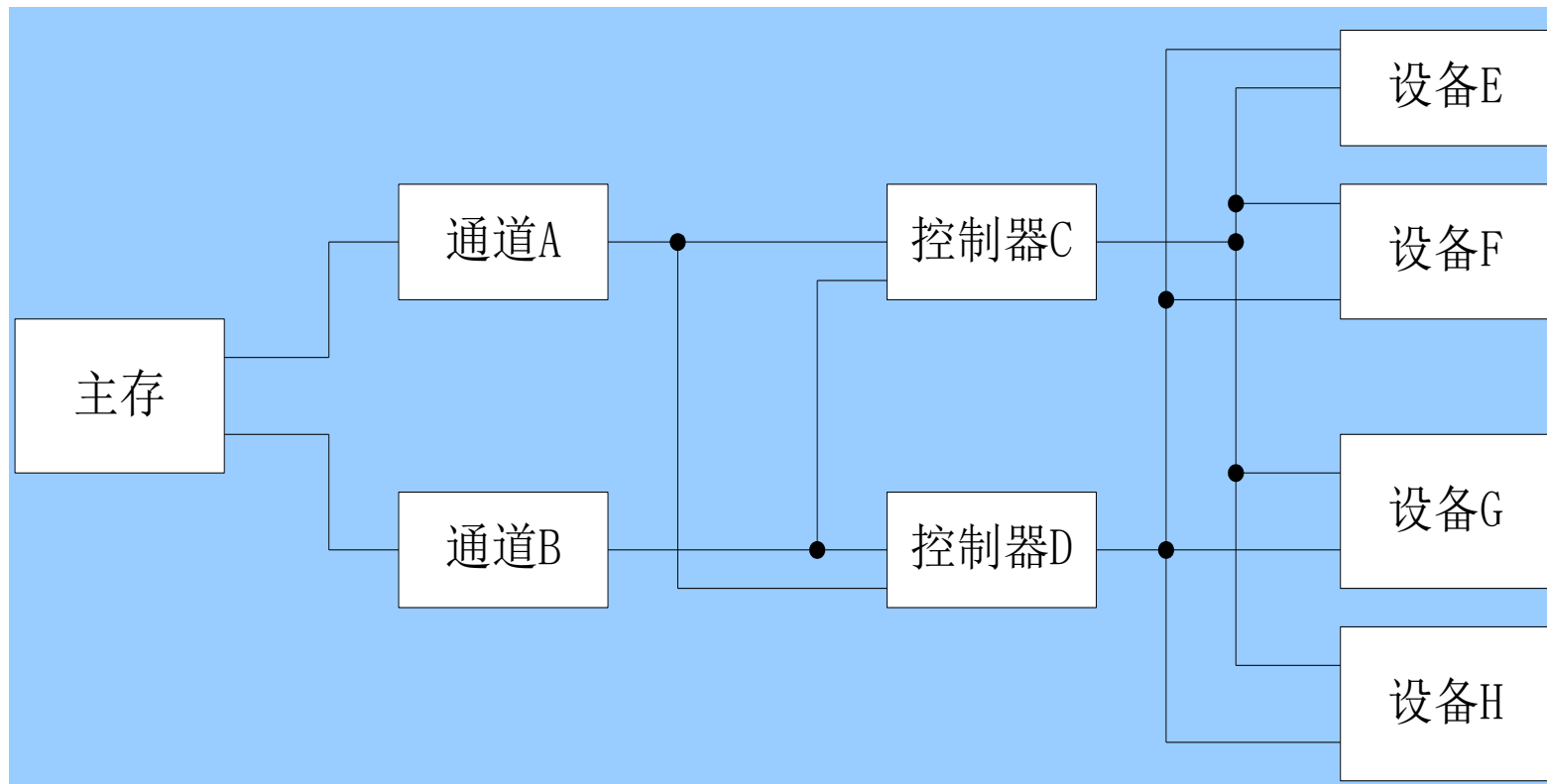
一个通道程序可以控制若干设备进行多次IO传输。

通道与DMA区别

- DMA方式需要CPU来控制传输数据库的大小、传输的内存位置，而通道方式中这些信息是由通道控制的
- 设置通道后，CPU只需向通道发出一条I/O指令。通道在收到该指令后，便在内存取出本次要执行的通道程序，然后执行该通道程序，仅当通道完成规定的I/O任务后，才向CPU发出中断信号
- 每个DMA控制器对应一台设备与内存传送数据，而一个通道可以控制多台设备与内存的数据交换



单通路I/O系统结构

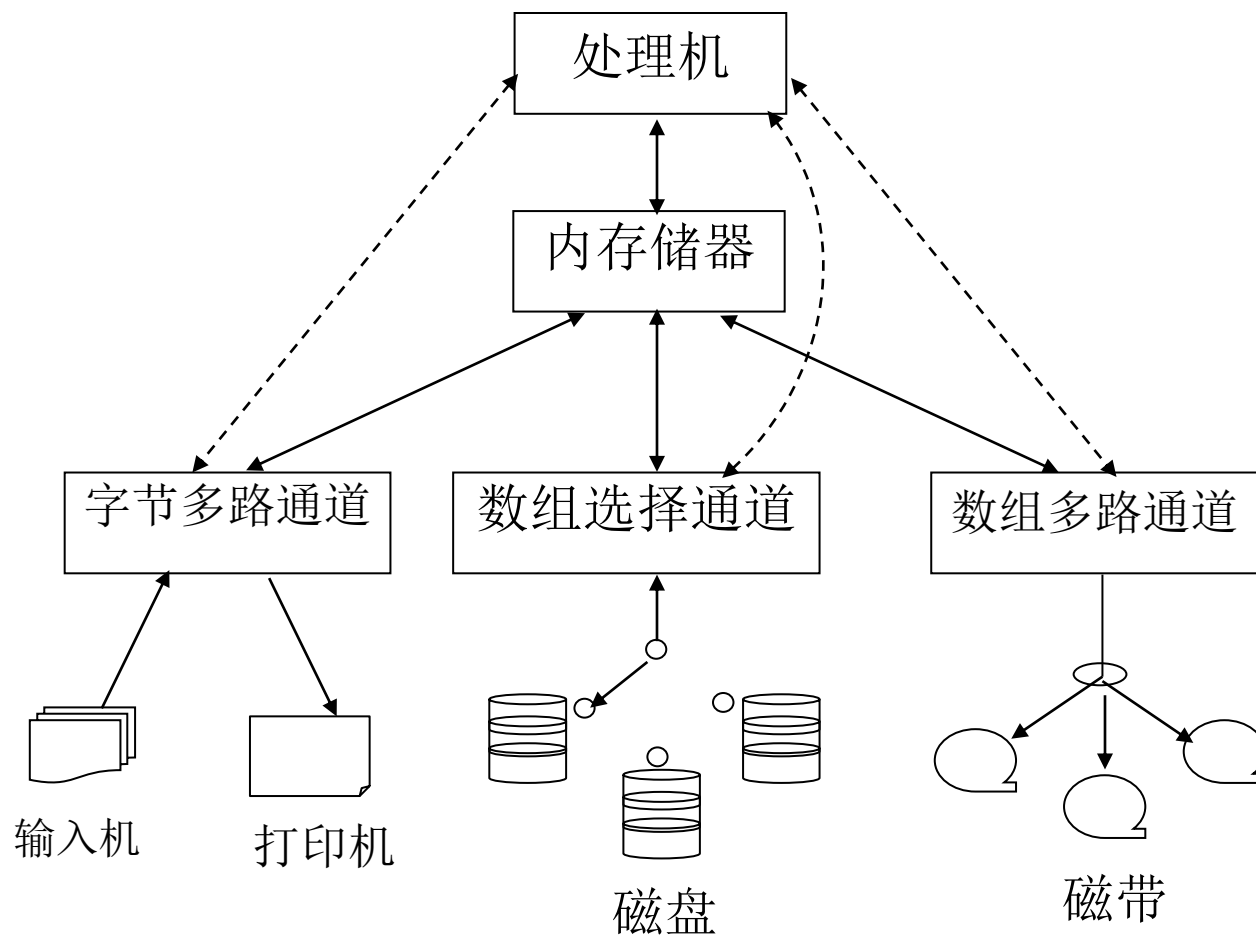


多通路I/O系统的配置方案

通道类型

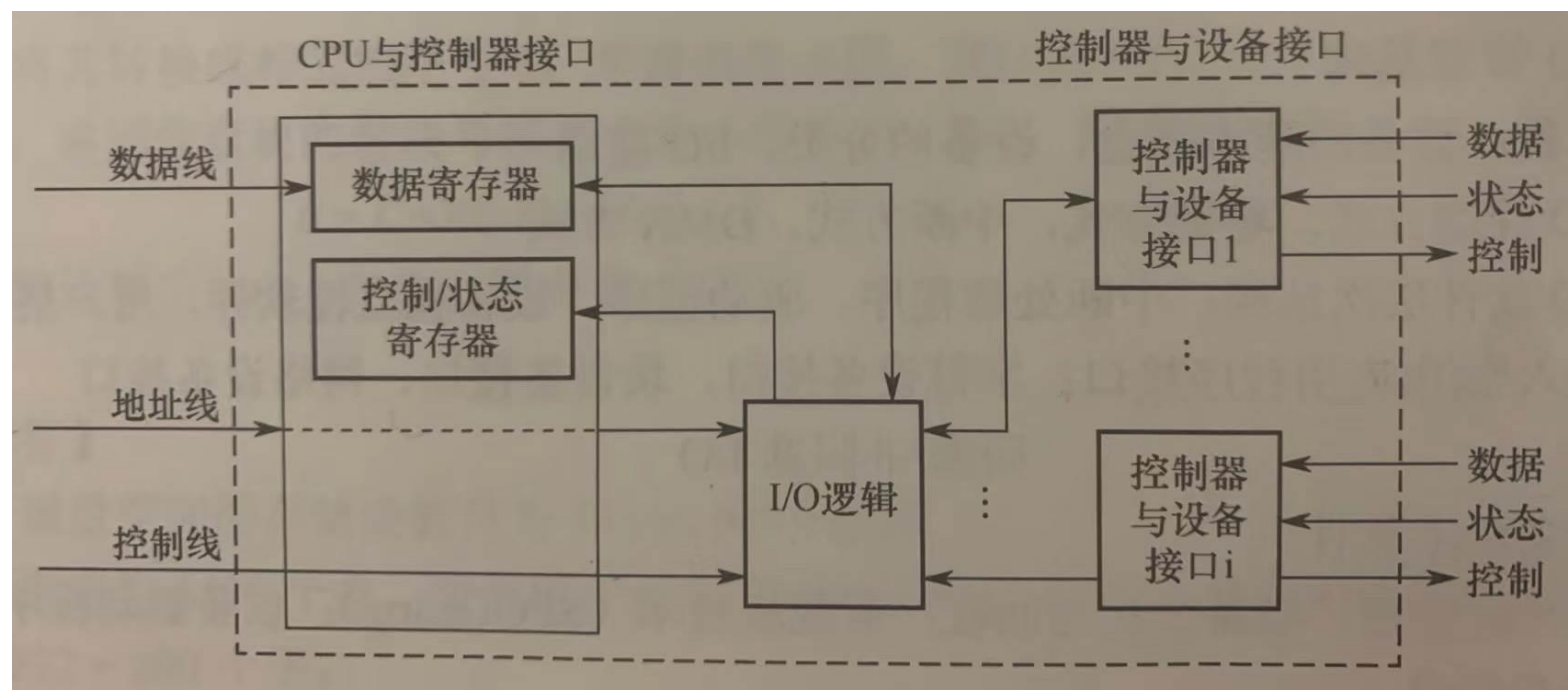
- 字节多路通道(byte multiplexer channel)
 - 以字节交换方式工作, 该通道有许多非分配型子通道, 其数量可达几十到几百个, 每个子通道连接一台I/O设备, 并控制该设备的I/O操作
 - 这些通道按时间片轮转方式共享主通道
 - 用于连接低速I/O设备
- 数组选择通道(block selector channel)
 - 以块为单位进行数据传输
 - 只含有一个分配型子通道, 在一段时间内只能执行一个通道程序, 控制一台设备进行数据传送, 直到该设备传送完毕释放该通道为止
 - 这种通道用于连接高速设备
- 数组多路通道(block multiplexer channel)
 - 以分时的方式执行几个通道程序, 它每执行一个通道程序的一条通道指令控制传送一组数据后, 就转向另一个通道程序
 - 这种通道用于连接高速设备

设备、通道、内存连接



I/O接口（设备控制器）

- 位于CPU与设备之间，既要与CPU通信，又要与设备通信，还要具有按CPU发来的命令去控制设备工作的功能。



设备控制器的组成

设备控制器功能

- 接收和识别CPU发来的命令，如磁盘控制器能接收读、写、查找等命令
- 数据交换，包括设备和控制器之间的数据传输，以及控制器和主存之间的数据传输
- 标识和报告设备的状态，以供CPU处理
- 地址识别
- 数据缓冲
- 差错控制

I/O端口

- I/O端口指设备控制器中可被CPU之间访问的寄存器，主要有3类寄存器
 - 数据寄存器：实现CPU与外设之间的数据缓冲
 - 状态寄存器：获取执行结果和设备的状态信息，以让CPU知道是否准备好
 - 控制寄存器：由CPU写入，以便启动命令或更改设备模式

- 采用通道实现I/O操作，数据输入过程
 - 当进程要求输入数据时，CPU向通道发出一条I/O指令，以给出要执行的通道程序的首地址和要访问的I/O设备
 - 对应通道接收到CPU发来的启动指令后，把存放在内存中的通道程序读出，并执行通道程序，控制设备将数据传送到内存中指定区域
 - 若数据传送结束，则向CPU发出中断请求信号。CPU收到中断信号后转至中断处理程序，**唤醒等待输入完成的进程**，并返回被中断程序
 - 在以后某个时刻，进程调度程序选中提出请求输入的进程，该进程从指定的内存单元中取出数据做进一步处理。

4种I/O控制方式总结

- 例 “一个客户要去裁缝店做一批衣服”
 - 客户相当于 “CPU”
 - 裁缝相当于 “设备”
 - DMA类秘书相当于 “DMA控制器”
 - 通道类秘书相当于 “通道”

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.5 I/O软件的层次

- I/O软件设计的基本思想
 - 设备管理软件组织成层次结构，向下与硬件有密切联系，向上与用户直接交互
 - 与进程管理、存储器管理、文件管理存在一定联系
 - 使复杂的I/O软件有清晰的结构，良好的可移植性和适应性
 - 分为4个层次

9.5 I/O软件的层次

- I/O 软件设计的基本思想是将设备管理软件组织成一种层次结构



9.5.1 用户I/O软件

- 实现与用户交互的接口，用户可直接调用在用户层提供的与I/O操作有关的库函数，对设备进行操作
- 操作系统的系统调用也是以函数界面形式供用户程序调用，如 Linux 的 `open()`、`read()`、`write()`、`close()` 等
- 这个层次的函数主要任务是为相应的系统调用提供参数，进而获取操作系统服务

9.5.2 与设备无关的系统软件

- 本层往下一一般在操作系统内核中实现。它对上层提供了系统调用的接口，对下通过设备驱动程序接口调用设备驱动程序
- 与设备无关的系统软件即系统调用处理程序
- 为了实现设备无关性，引入了逻辑设备和物理设备概念
- 在系统实际执行时，必须将逻辑设备名映射成物理设备名使用

9.5.2 与设备无关的系统软件(Cont.)

- 主要功能
 - (1) 向设备驱动程序提供统一接口
 - 将逻辑设备名映射为物理设备名
 - (2) 向用户层提供统一接口
 - (3) 设备保护，禁止用户直接访问设备
 - (4) 缓冲，缓冲区通常在系统区
 - (5) 提供与设备无关的块尺寸
 - (6) 错误报告
 - (7) 设备的分配与去配

9.5.3 设备驱动程序

- 设备驱动程序是I/O进程与设备控制器之间的通信程序
 - 将用户提交的逻辑I/O请求转化为物理I/O操作的启动和执行
 - 具体实现系统对设备发出的操作指令，驱动I/O设备工作
- 响应通道发来的中断请求，根据中断类型调用相应中断处理程序
- 设备驱动程序与硬件直接有关，具体包括：构造通道程序，启动设备进行数据传输，响应通道发来的中断请求

9.5.3 设备驱动程序（Cont.）

■ 主要功能

- ① 将接收到的上层软件发来的抽象要求转换为具体要求
- ② 检查用户 I/O 请求的合法性，了解 I/O 设备的状态，传递有关参数，设置设备的工作方式
- ③ 发出 I/O 命令，启动分配到的 I/O 设备，完成指定的 I/O 操作及时响应由控制器或通道发来的中断请求，并根据其中断类型调用相应中断处理程序进行处理
- ④ 对于设置有通道的计算机系统，驱动程序还应能够根据用户的 I/O 请求，自动地构成通道程序

9.5.3 设备驱动程序（Cont.）

- 设备驱动程序的处理过程（以磁盘为例）
 - 计算请求块在磁盘的实际位置
 - 检查驱动器的电机是否运转，确定磁头引臂是否定位在正确的柱面上。。。
 - 向控制器发出相应命令
 - 操作完成后检查是否有错，一切正常则负责将数据传送到与设备无关的软件层
 - 向调用者返回错误报告的状态信息
 - 若还有其它未完成的请求在排队，则选择一个启动

9.5.4 中断处理程序

- 用于保存被中断进程现场，转入相应的中断处理程序进行处理
- 处理完后恢复现场，并返回到被中断的进程

- 例：当用户程序想要读取文件中的数据时，执行步骤：
 - 用户I/O软件对用户的I/O请求进行解释，假设调用read接口
 - 与设备无关的系统软件首先在高速缓存中查找该数据块
 - 若未找到，则调用设备驱动程序向硬件发出相应请求，用户进程阻塞直到数据块被读出
 - 当磁盘操作完成，硬件产生一个中断，并转入中断处理程序
 - 中断处理程序检查中断原因，唤醒被阻塞进程以结束此次I/O 请求，使用户进程继续执行

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

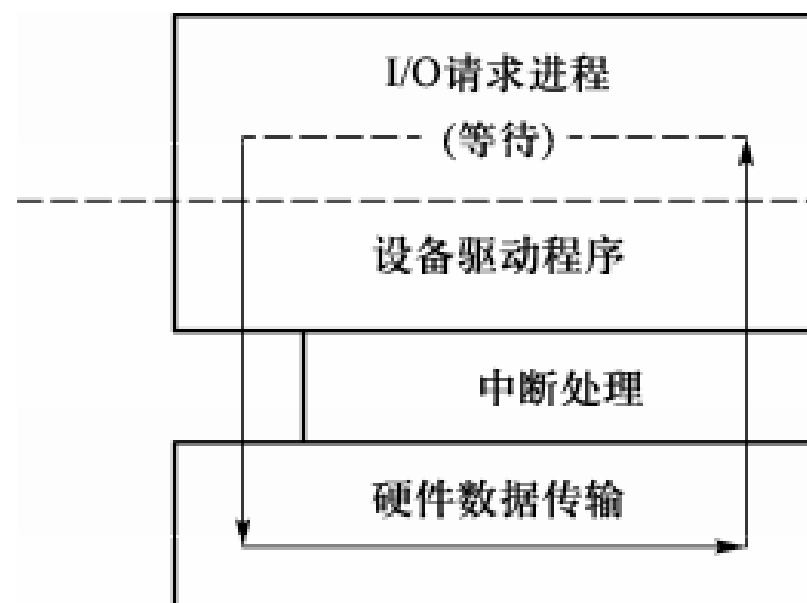
9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

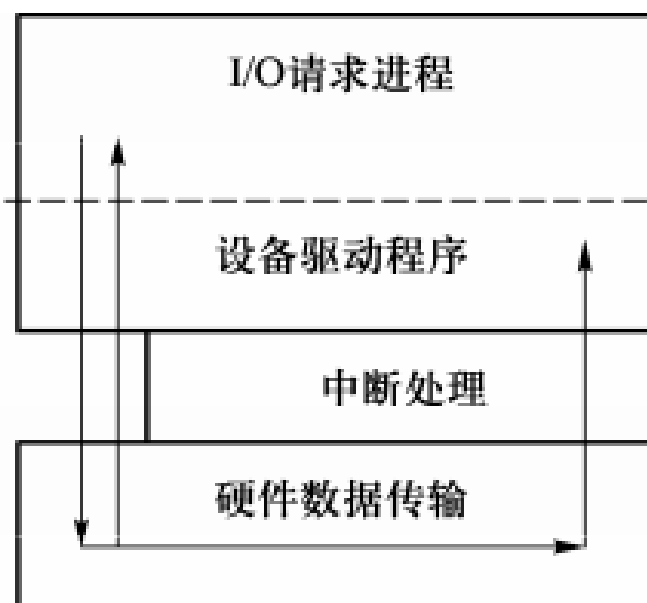
9.15 系统举例-Linux

9.6 5 同步I/O与异步I/O



时间 →

(a) 同步I/O



时间 →

(b) 异步I/O

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.7 设备的分配与去配

- 独占型设备的分配与去配
 - 块型独占
 - 字符型独占
- 共享型设备的分配与去配
 - 块型共享

数据结构

设备控制块 (UCB)

设备标识
设备状态
相连通道
占有设备进程

控制器控制块 (CUCB)

控制器标志
控制器状态
相连通道
占有控制器进程

通道控制块 (CCB)

通道标识
通道状态
通道类型
占有通道进程

系统设备表(SDT)



独占型设备的分配与去配

用户使用独占型设备活动：

申请，使用，使用，…，使用，释放

申请：(1) 根据设备类查SDT表

(2) $P(S_m)$

(3) 查UCB表找一空闲设备并分配

使用：(1) 分配通道、分配控制器

(2) 形成通道程序，启动通道，…，通道中断处理

(3) 去配通道

释放：(1) 访问SDT表

(2) 查UCB表项，去配

(3) $V(S_m)$

9.7 设备的分配与去配

■ 共享型设备的分配与去配

- 用户使用共享型设备活动
 - 使用，使用，...，使用
- 特征
 - 来自文件系统，或虚拟存储管理系统，输入井/输出井；
 - 每次读（写）一块；
 - 通常经过缓冲；
 - 排队优化。

9.7 设备的分配与去配（Cont.）

- 使用（有通道形式）
 - 分配通道；
 - 形成通道程序，启动通道，通道控制设备进行操作；
 - 中断处理；
 - 释放通道
- 使用（无通道形式）
 - 分配缓冲，填写头部，
 - 若是写，信息复制到缓冲区，缓冲区链入IO链尾部，若设备空闲启动设备，IO结束时释放缓冲区；
 - 若是读，缓冲区链入IO链尾部，若设备空闲启动设备，等待（读入数据），中断唤醒后，信息复制到进程空间，释放缓冲区。

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

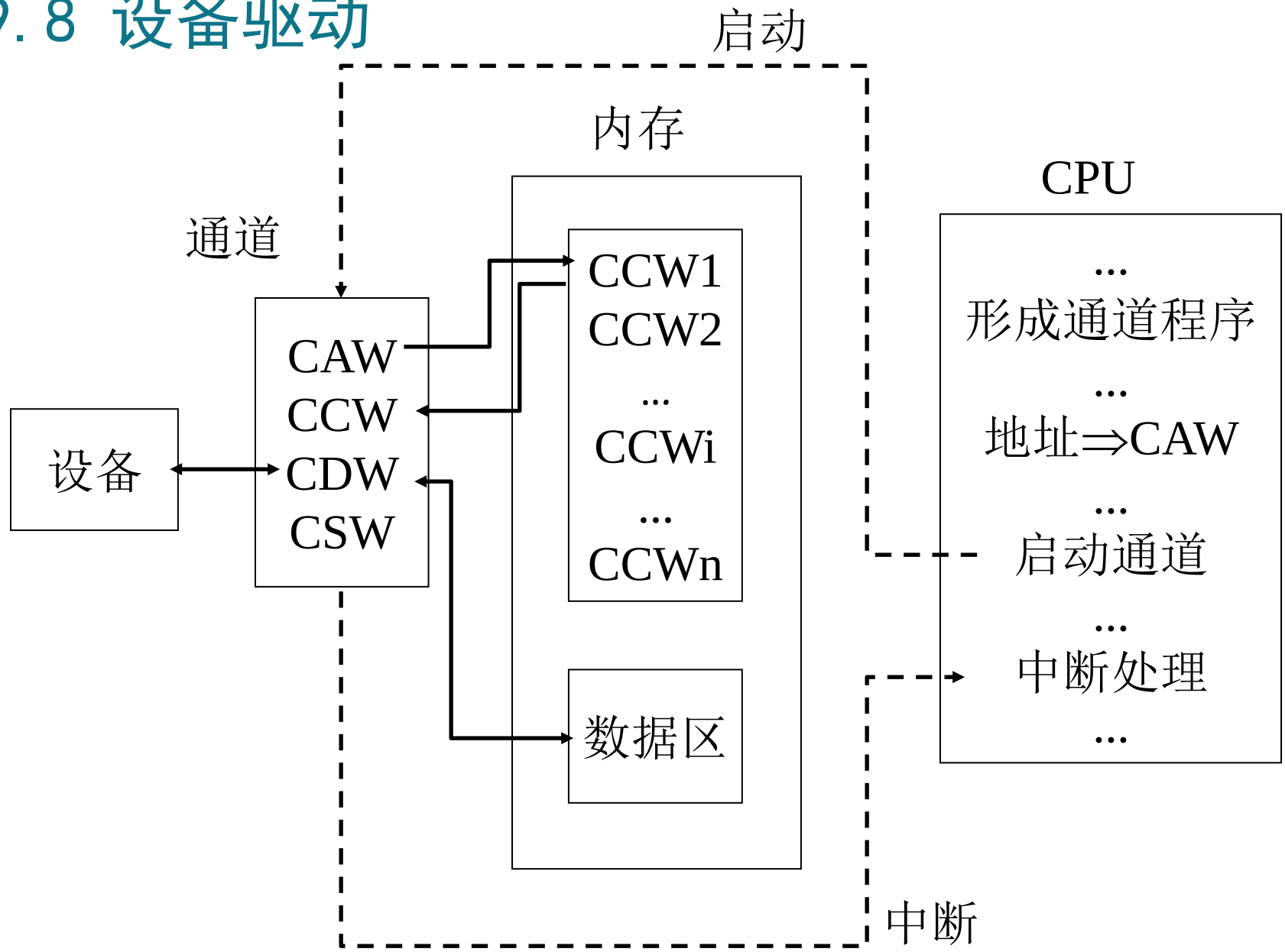
9.8 设备驱动

- 设备驱动程序的特点
 - 设备驱动程序与硬件密切相关
 - 每类设备都要配置特定的驱动程序
 - 驱动程序一般由设备厂商根据操作系统要求编写
 - 操作系统仅对与设备驱动的接口提出要求

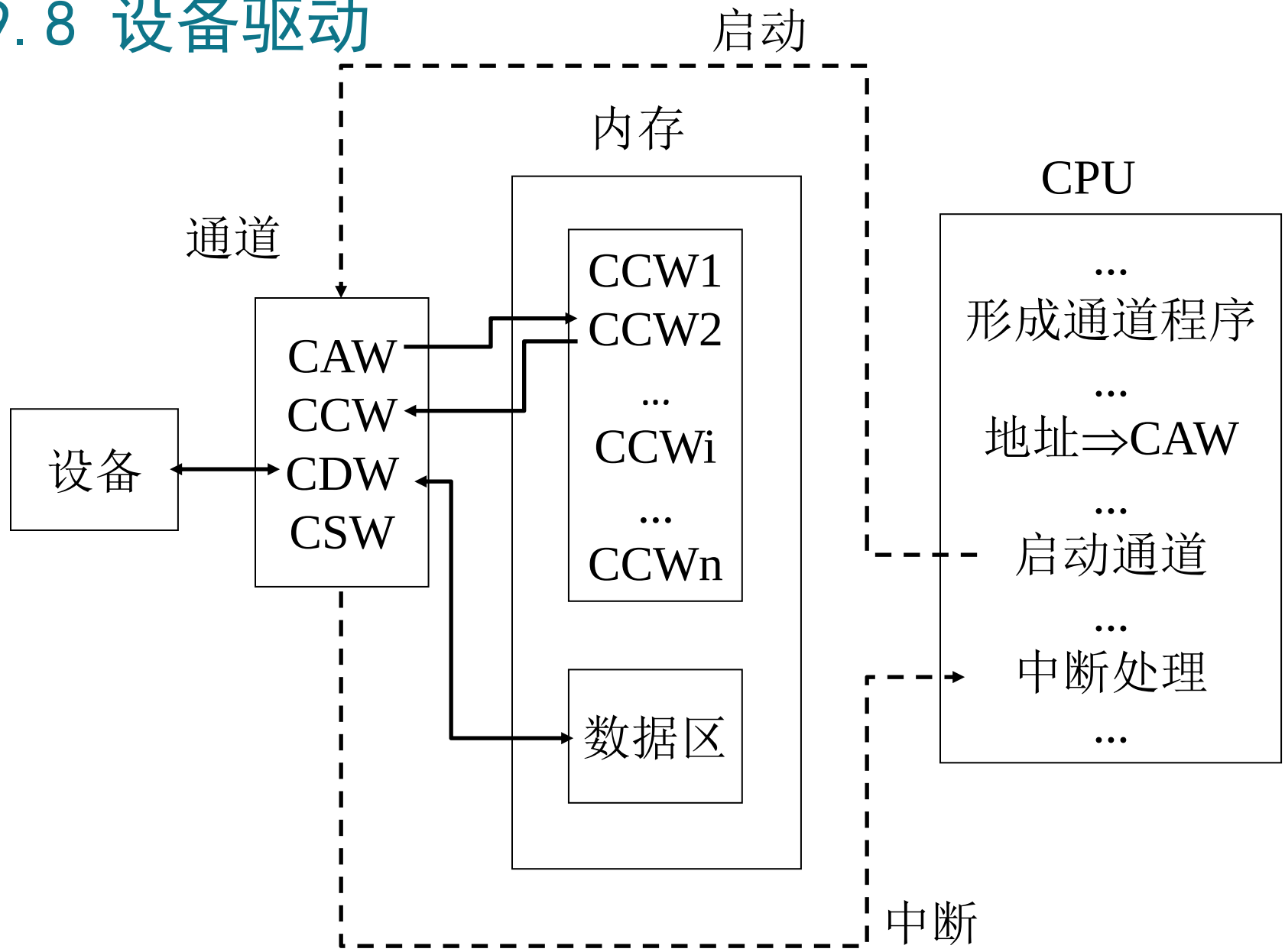
9.8 设备驱动（Cont.）

- 通道程序
 - CCW指令序列
 - 静态编制或动态生成
 - 多通道可并行执行相应通道程序
- 设备启动
 - 通道程序形成后，CPU将通道程序的起始地址放到内存的指定单元处，然后执行通道启动指令使通道开始工作
 - 通道启动后，由指定单元取出通道程序起始地址，放入通道地址字中，由此依次地执行各种通道指令
- 中断处理
 - 通道向CPU发的中断

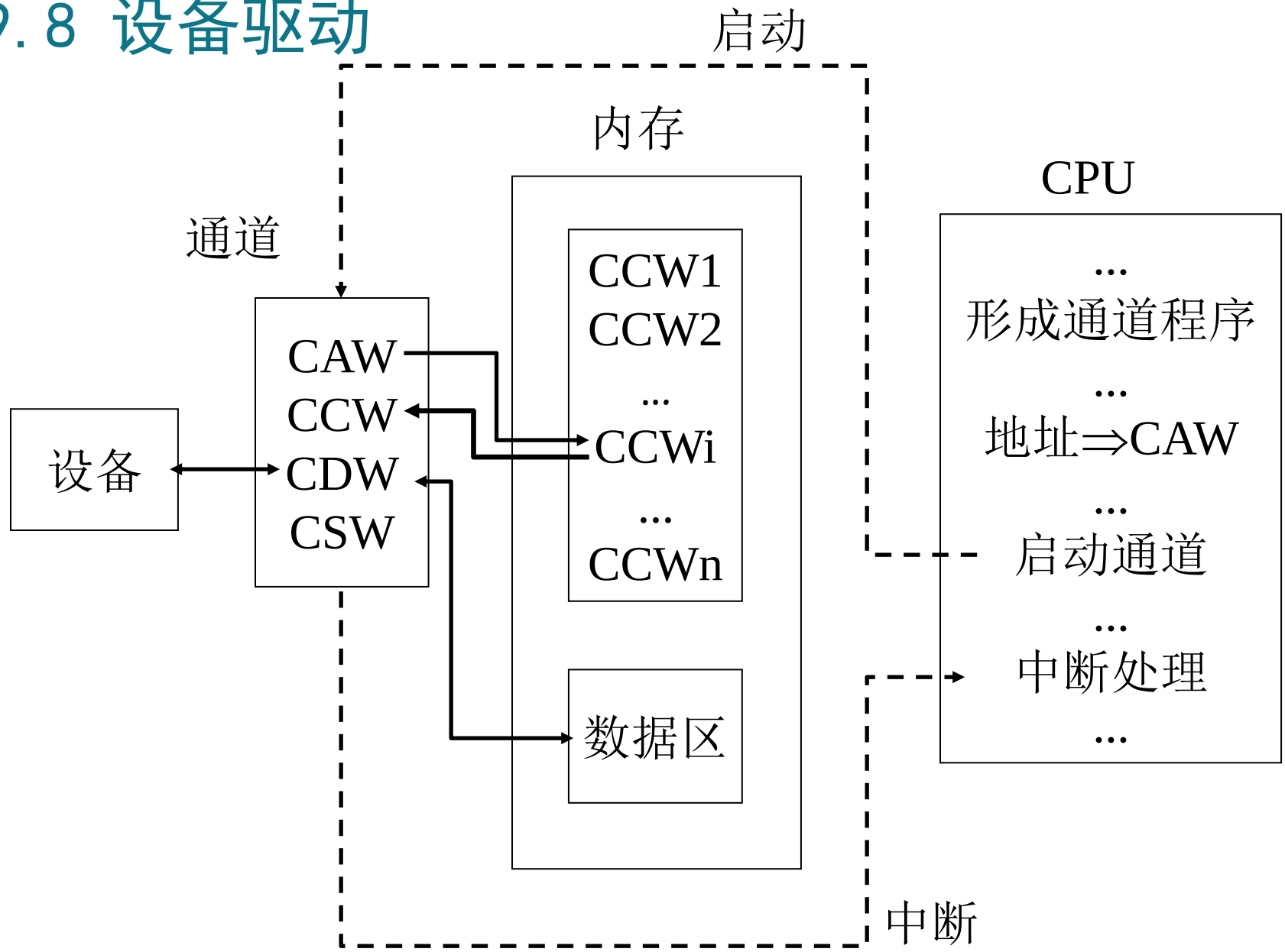
9.8 设备驱动



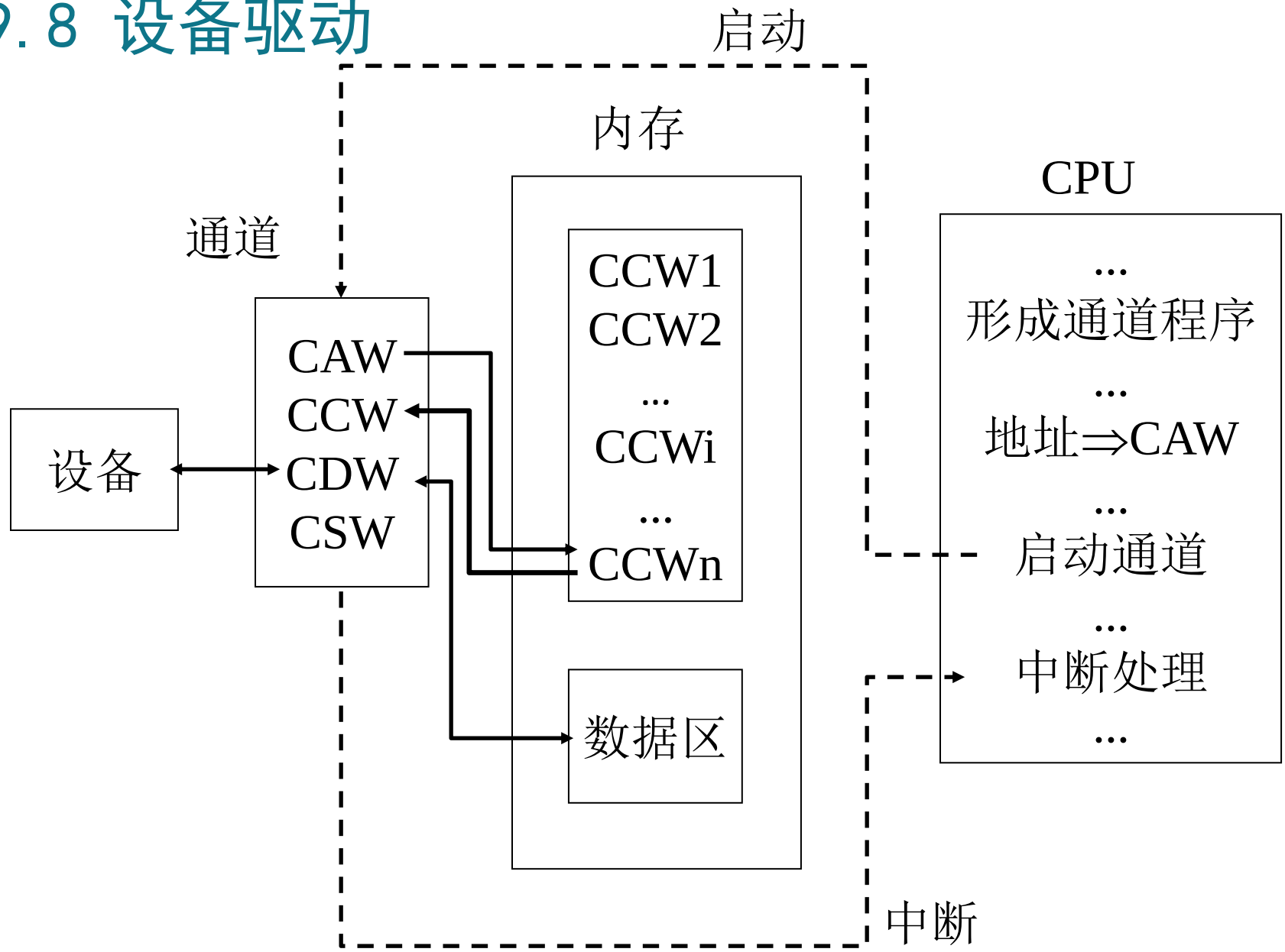
9.8 设备驱动



9.8 设备驱动



9.8 设备驱动



提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.9 设备调度

- 优化服务顺序
 - 寻道时间
 - 旋转延迟时间
 - 传输时间
- 考虑因素
 - 公平性：防止饿死
 - 高效性：减少磁盘引臂移动量，减少旋转延迟

9.9.1 磁盘I/O参数

- 读/写一个磁盘块需要多少时间。由三个因素确定：
 - 寻道时间 (seek time)：将磁盘引臂移动到指定柱面所需要的时间；
 - 旋转延迟 (rotational delay)：指定扇区旋转到磁头下的时间；
 - 传输时间 (transfer time)：读/写一个扇区的时间。

9.9.1 磁盘I/O参数 (Cont.)

- 寻道时间 T_s 计算公式如下：
 - $T_s = m \times n + s$
 - 其中， n 为跨越磁道数， m 为跨越一个磁道所用时间， s 为启动时间。
- 旋转延迟 T_r 计算公式如下：
 - $T_r = 1 / (2r)$
 - 其中， r 为磁盘转速。
- 传输时间 T_t 计算公式如下：
 - $T_t = b / (rN)$
 - 其中， b 为读/写字节数， r 为磁盘转速， N 为一条磁道上的字节数

9.9.1 磁盘I/O参数 (Cont.)

- 因此，可将访问时间 T_a 表示为：

$$T_a = T_s + T_r + T_t = m \times n + s + \frac{1}{2r} + \frac{b}{rN}$$

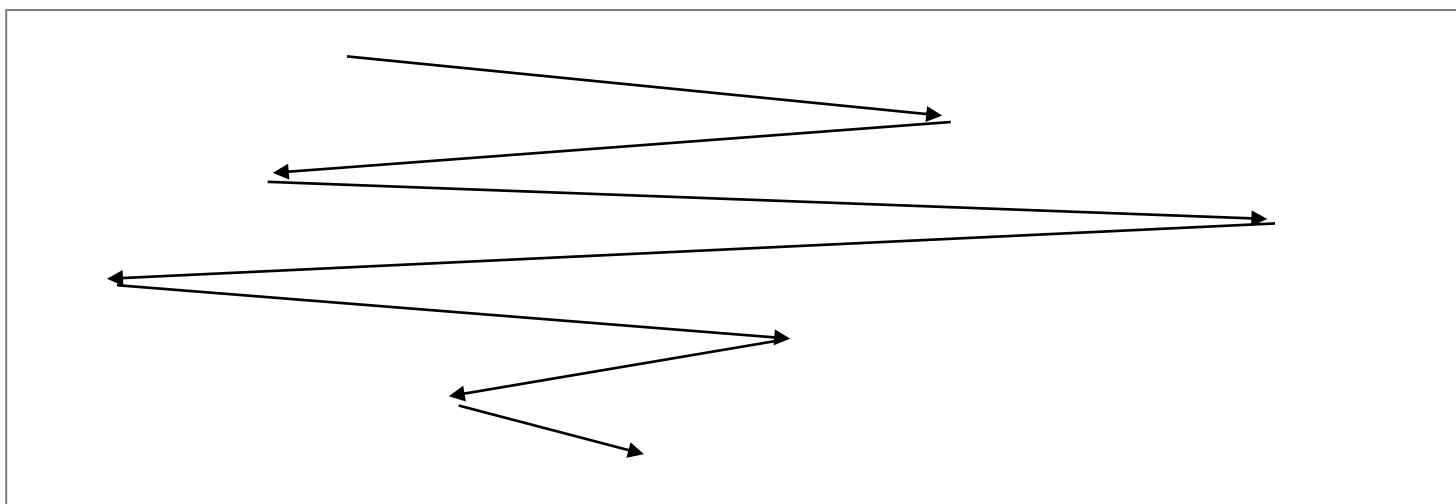
- 访问磁盘通常是以扇区（块）为单位的，令 M 为一个磁道上扇区的个数，则一个扇区的访问时间为：

$$T_a = T_s + T_r + T_t = m \times n + s + \frac{1}{2r} + \frac{1}{rM}$$

磁头引臂调度 (disk head scheduling)

- 先到先服务 (FCFS) : 按照输入输出请求的次序为各个进程服务。
 - 请求序列: 130, 42, 180, 15, 108, 68, 97

0 15 42 53 68 97 108 130 180 199



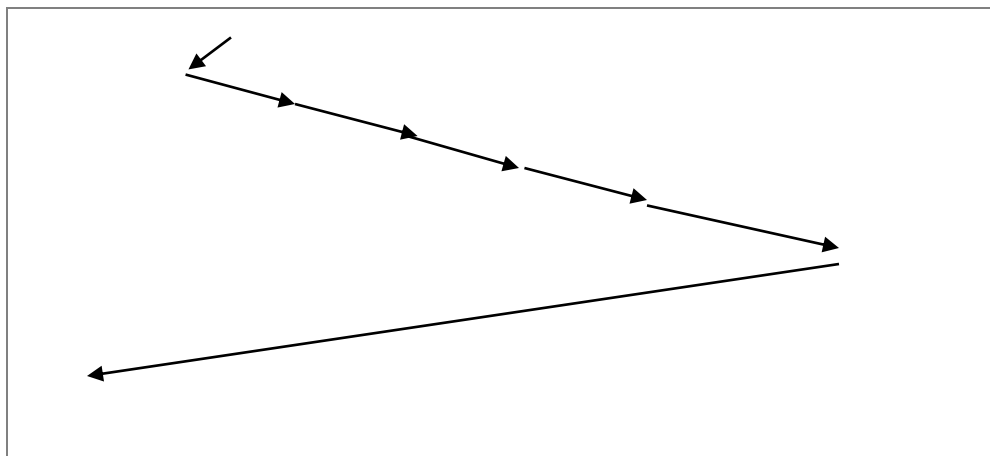
移动量: $(130-53) + (130-42) + (180-42) + (180-15) + (108-15) + (108-68) + (97-68) = 630$

- 特点: 公平、简单, 但执行效率不高

磁头引臂调度 (disk head scheduling)

- 最短寻找时间优先 (SSTF)：优先为距离磁头当前所在位置最近柱面的请求服务。
- 请求序列：130, 42, 180, 15, 108, 68, 97

0 15 42 53 68 97 108 130 180 199



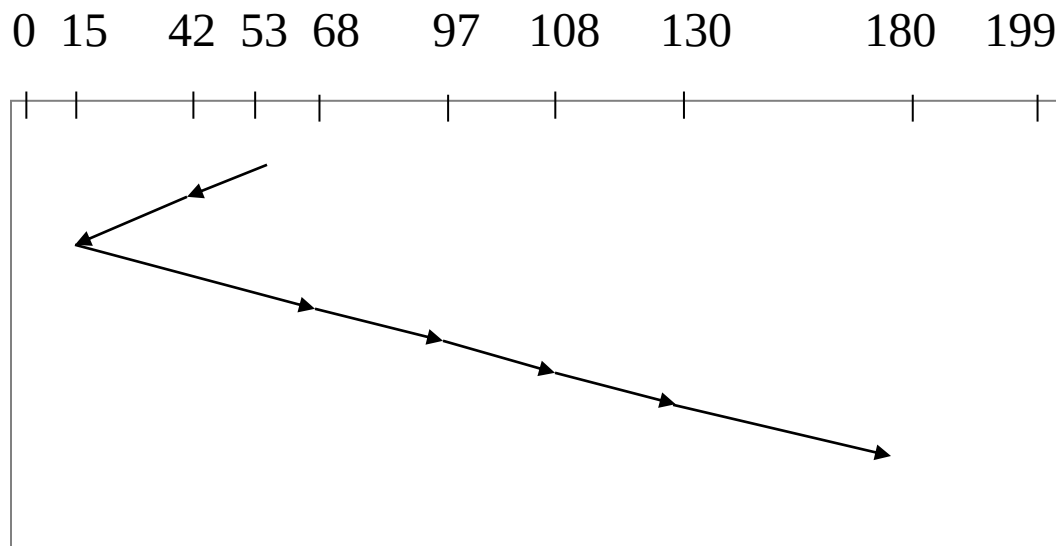
移动量： $(53-42) + (180-42) + (180-15) = 314$

优点：效率高

缺点：磁道歧视

磁头引臂调度 (disk head scheduling)

- 扫描算法（电梯算法）：无访问请求时，磁头引臂停止不动；当有访问请求时，引臂按照电梯移动规律运动，并为路经柱面的访问请求服务。
 - 请求序列：130, 42, 180, 15, 108, 68, 97



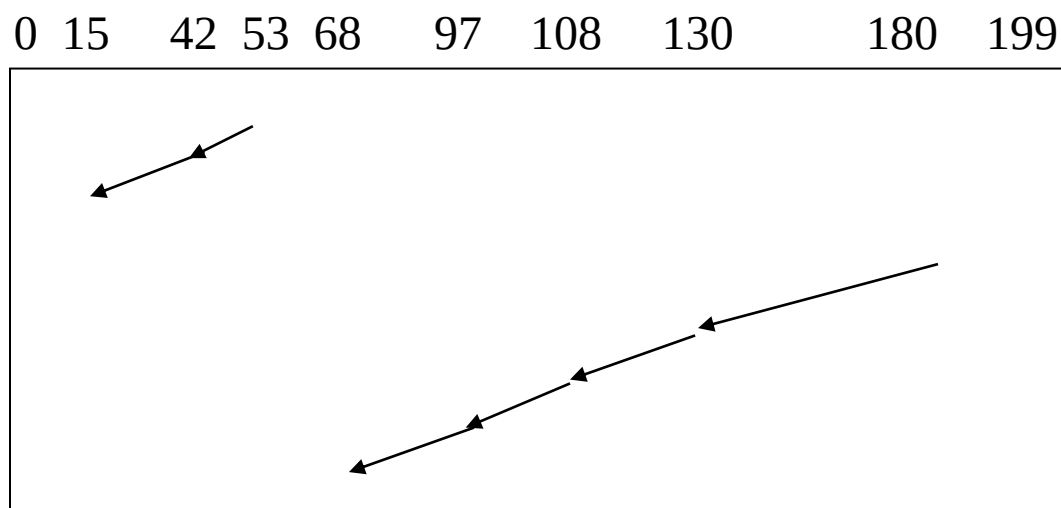
移动量： $(53-15) + (180-15) = 203$

优点：公平，效率高

缺点：存在磁道地域差别

磁头引臂调度 (disk head scheduling)

- 循环扫描算法：磁头仅在单方向移动过程中才为路经的请求服务，但只是移动到最内（外）请求柱面为止。
 - 请求序列：130, 42, 180, 15, 108, 68, 97



特点：所有磁道地位相同，最长等待时间相同

优点：高效

问题：diskhead stickiness (磁头粘性)

缺点：磁头黏性

磁头引臂调度 (disk head scheduling)

- N-step SCAN (N步扫描)
 - 将磁盘请求队列分为若干个长度为N的子队列，每个队列内采用SCAN算法
 - 例子：磁道由外向内编号0-99，磁头当前位置20，向内移动，N=4
 - 12, 5, 7, 30, 60, 77, 13, 26, 61, 80, 53, 66
 - 20 → 30 → 12 → 7 → 5 → 13 → 26 → 60 → 77 → 80 → 66 → 61 → 53
 - 当N很大时，接近SCAN算法
 - 当N=1时，蜕化为FCFS算法

磁头引臂调度(disk head scheduling)

- FSCAN (Freezing SCAN, 冻结扫描)
 - 将磁盘请求分为两个子队列
 - 服务队列
 - 请求队列
 - 用SCAN算法扫描服务队列，并为请求服务，服务期间新到达的请求入请求队列
 - 扫描完成后交换两个队列的地位
 - 特点：克服磁道粘性，防止饿死

例题

- 例：设有一个只有一个移动磁头的磁盘，磁道由外向内编号0、1、2、...、199，磁头移动一个磁道所需时间为1ms，每个磁道有100个扇区，磁盘转速6000r/m。采用CLOOK引臂调度算法，当前引臂位置处于第100磁道，当前移动方向由外向内，并规定引臂向内扫描时为路经请求服务。对于如下磁道请求120、85、70、30，每个请求访问对应磁道上的一个扇区，问：
 - (1) 给出引臂移动序列，计算引臂移动量和寻道时间，忽略启动时间；
 - (2) 计算平均旋转延迟时间；
 - (3) 计算传输时间；
 - (4) 计算所有访问处理时间。

■ 解答：

(1) 磁盘引臂移动序列为：100→120→30→70→85，跨越磁道数：20+90+40+15=165。共需寻道时间
 $165 \times 1\text{ms} = 165\text{ms}$ 。

(2) 1次访盘的旋转延迟为：
 $T_r = 1 / (2r) = 1 / (2 \times (6000/m)) = 1 / (2 \times (100/s)) = 5\text{ms}$ ，4次访盘的旋转延迟为 $4 \times 5\text{ms} = 20\text{ms}$ 。

(3) 1次访盘的传输时间为：
 $T_t = 1 / (rM) = 1 / ((6000/m) \times 100) = 1 / ((100/s) \times 100) = 0.1\text{ms}$ ，4次访盘的传输时间为 $4 \times 0.1\text{ms} = 0.4\text{ms}$ 。

(4) 所有访问处理时间 = $165 + 20 + 0.4 = 185.4\text{ms}$ 。

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.10 缓冲技术

- 引入缓冲的原因
 - 缓和CPU与I/O设备间速度不匹配的矛盾
 - 减少对CPU的中断频率，放宽对中断响应时间的限制
 - 提高CPU和I/O设备之间的并行性

高速缓存

- 操作系统通过磁盘高速缓存技术提高磁盘的I/O速度，对访问高速缓存要比访问原始磁盘数据更高效
 - 例如：正在运行进程的数存据既存储在磁盘上，又存储在内存。

9.10.1 缓冲技术（Cont.）

- Buffering vs. Caching
 - buffering: 处理数据到达与离开速度不一致所采用的技术。
 - caching: 将慢速存储器上活动信息缓冲到快速存储设备上的技术。
- 硬缓冲与软缓冲
 - 硬缓冲区通常设在设备中
 - 软缓冲区通常设在内存系统空间中（操作系统管理）
- 私用缓冲与公共缓冲
 - 私用缓冲: 一个缓冲区与一个固定设备相联系，不同设备使用不同的缓冲区
 - 利用率低
 - 公用缓冲: 缓冲区由系统统一管理，按需要动态分派给正在进行I/O传输的设备

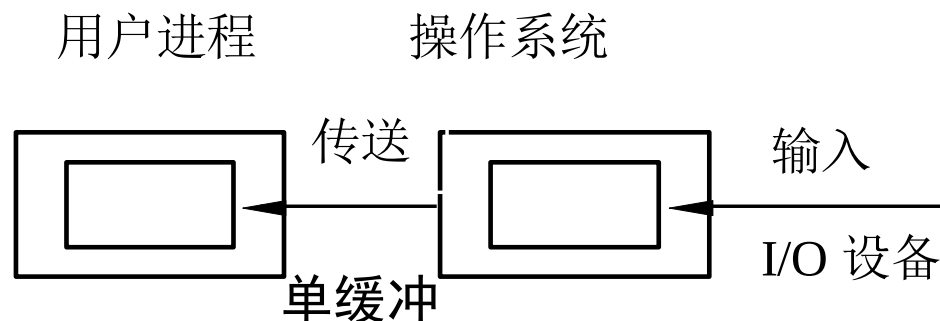
9.10 缓冲技术（Cont.）

- 公共缓冲的管理
 - 单缓冲
 - 双缓冲
 - 循环缓冲
 - 缓冲池
- 缓冲区的分类
 - 字符型缓冲
 - 块型缓冲

9.10 缓冲技术 (Cont.)

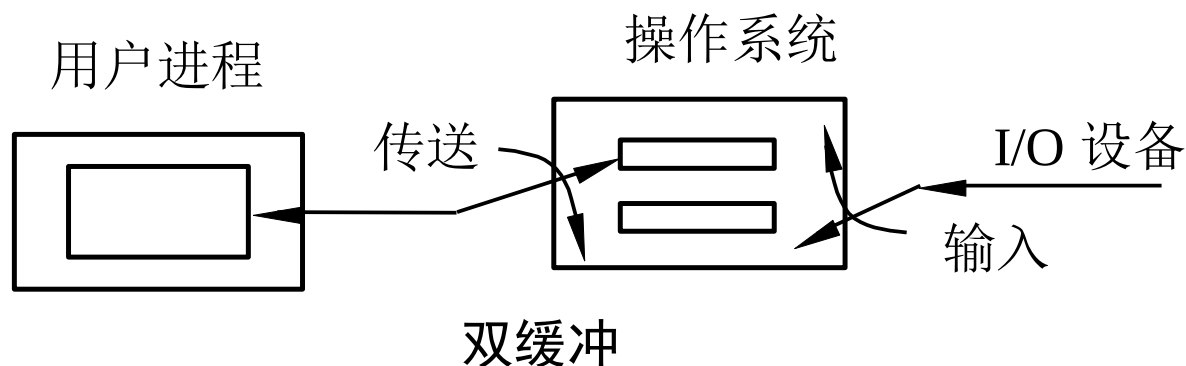
1. 单缓冲

- 当用户进程发出一个I/O请求时，OS便在主存中分配一个缓冲区。
- 设备与进程交换数据（如输入）时信息由设备传到缓冲区，然后再由缓冲区传给进程。
- 设备与处理器对缓冲区的操作是串行的



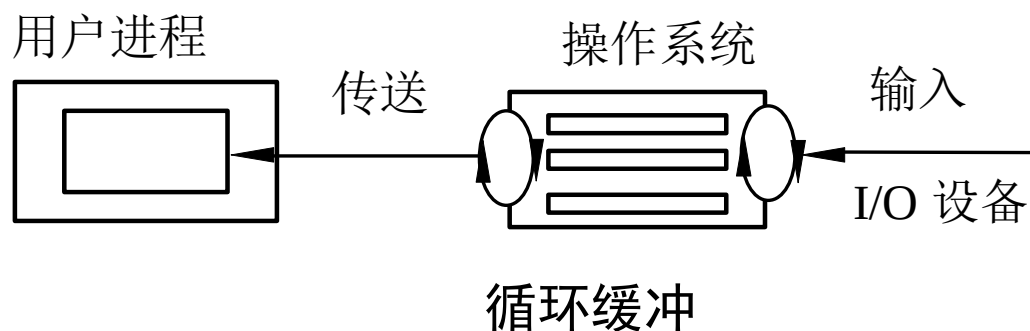
2. 双缓冲

- 双缓冲提供两个缓冲区。
 - 进程和设备传送数据时（输入），信息由设备传送到一个缓冲区
 - 如满后，信息由设备传到第二个缓冲区，**同时处理器将第一个缓冲区内容复制到进程空间**
 - 然后，再将输入信息传到第一个缓冲区
 - **如此交替**，可以提供设备和CPU之间的并行性。
- 但双缓冲只是一种说明设备与设备、CPU与设备并行操作的简单模型，并不适用于实际系统中的并行操作。



3. 循环缓冲

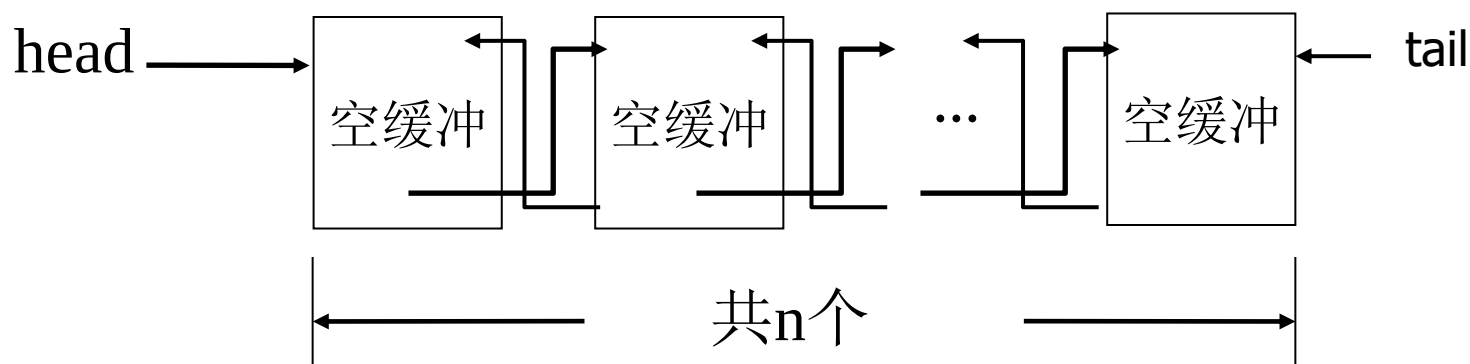
- 由于双缓冲并不能真正解决实际系统中的并行操作，于是引入了多缓冲。即在设备与进程之间设置多个缓冲区，这些缓冲区连成环状。
- 有两个指针 in 和 out，in 指向当前输入的位置，out 指向当前取出的位置。
- 通过增加缓冲区的个数，可使并行程度得到明显提高。
- 循环缓冲包括以下两部分。
 - (1) 多个缓冲区
 - (2) 多个指针



4. 缓冲池

- 由多个系统公用的缓冲区组成
- 缓冲池的结构: 相同类型的缓冲区链成一个队列
 - 一组用于块型设备, 其长度通常与外围设备物理块的长度相同
 - 一组用于字符型设备, 其长度通常为8 B、16 B 等。
- 缓冲池的缓冲区按其使用情况, 可以形成三个队列
 - 空缓冲队列
 - 输入队列
 - 输出队列

缓冲池管理



Var buf_num:semaphore; (init n)

mutex:semaphore; (init 1)

1. 申请

(1) P(buf_num)

(2) P(mutex)

(3) 取链头空缓冲

(4) V(mutex)

2. 释放

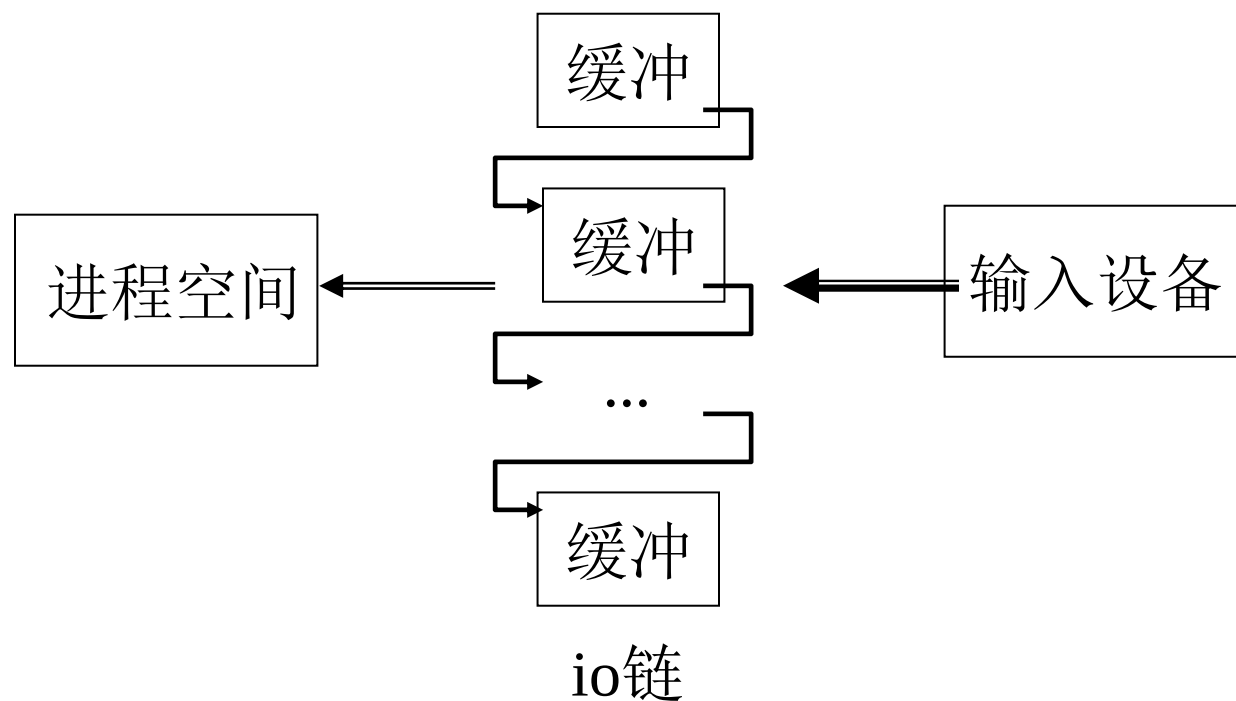
(1) P(mutex)

(2) 空缓冲入链尾

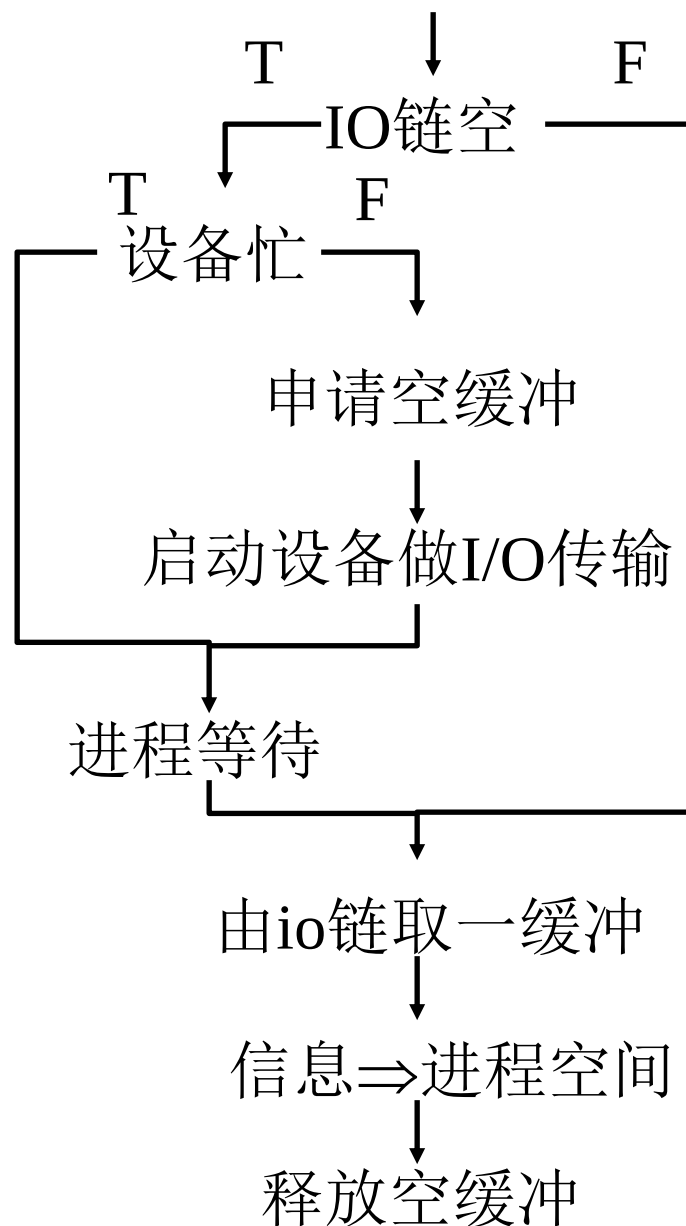
(3) V(mutex)

(4) V(buf_num)

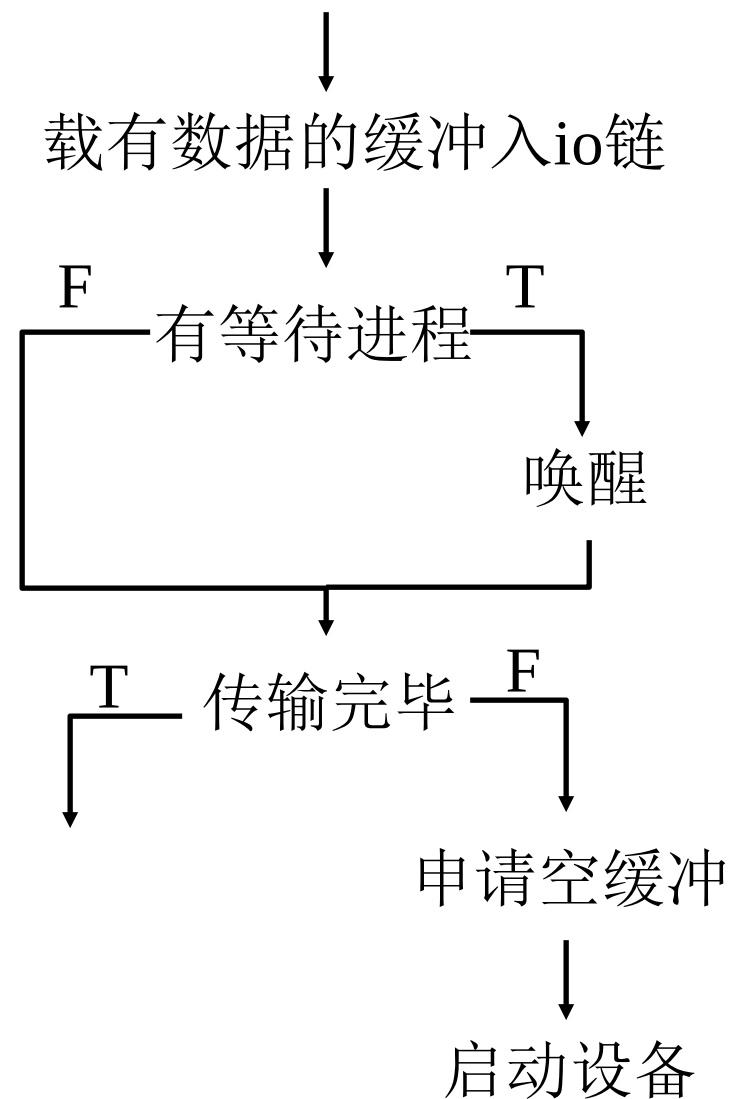
9.10.6 缓冲技术实现：输入设备



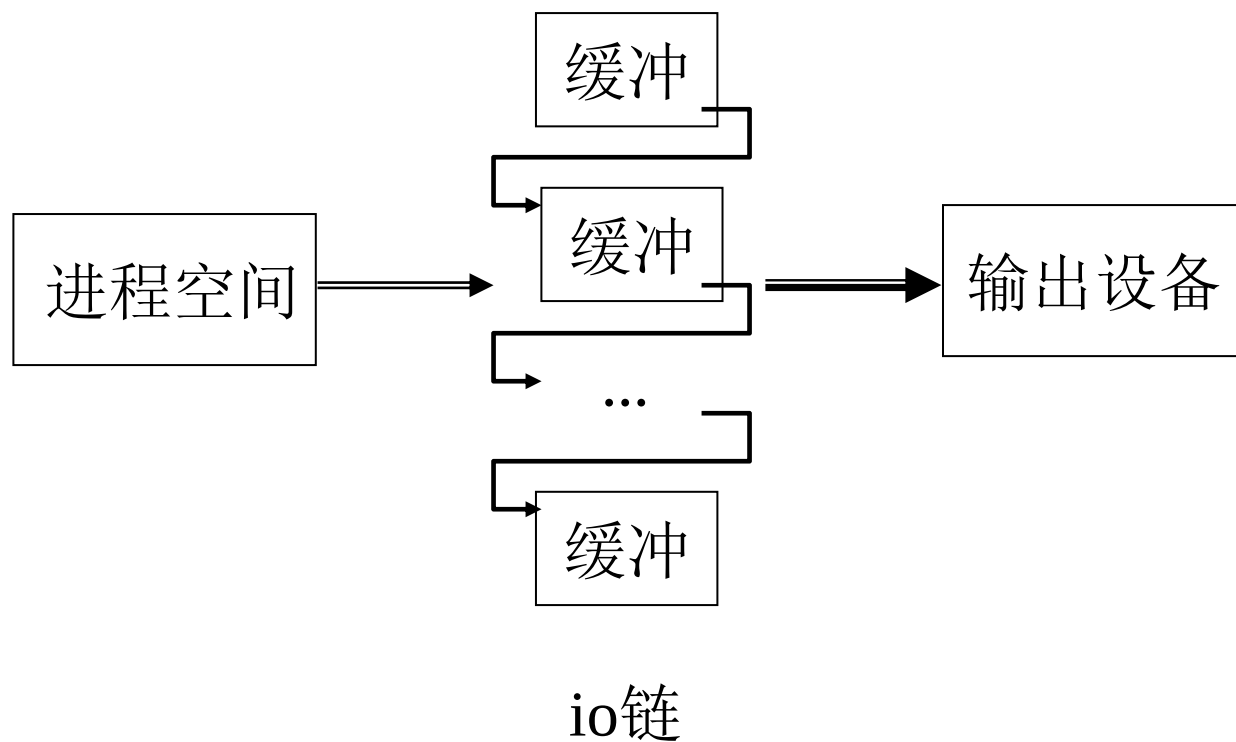
进程方面:



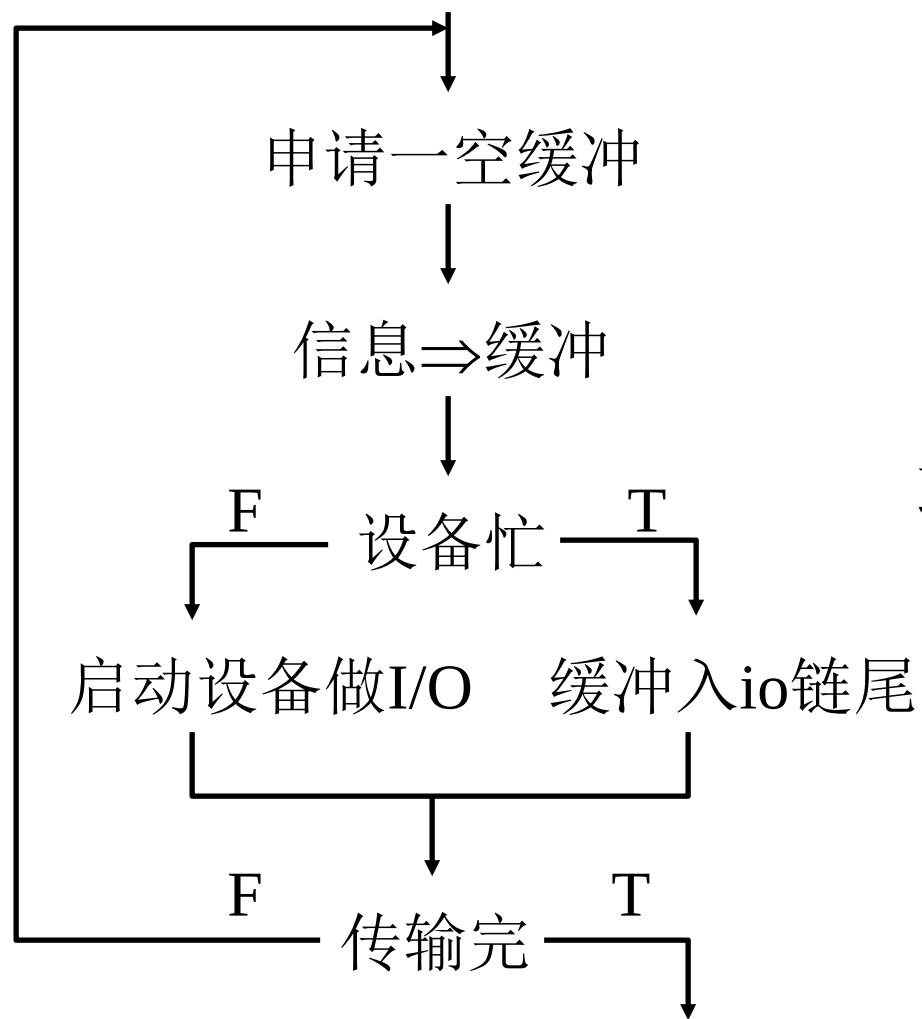
中断方面:



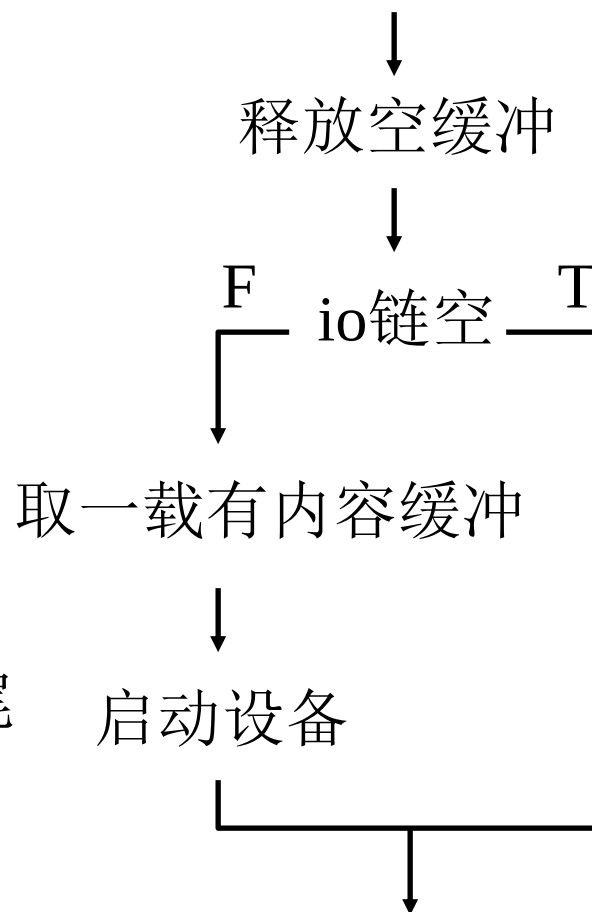
9.10.6 缓冲技术实现：输出设备



进程方面：



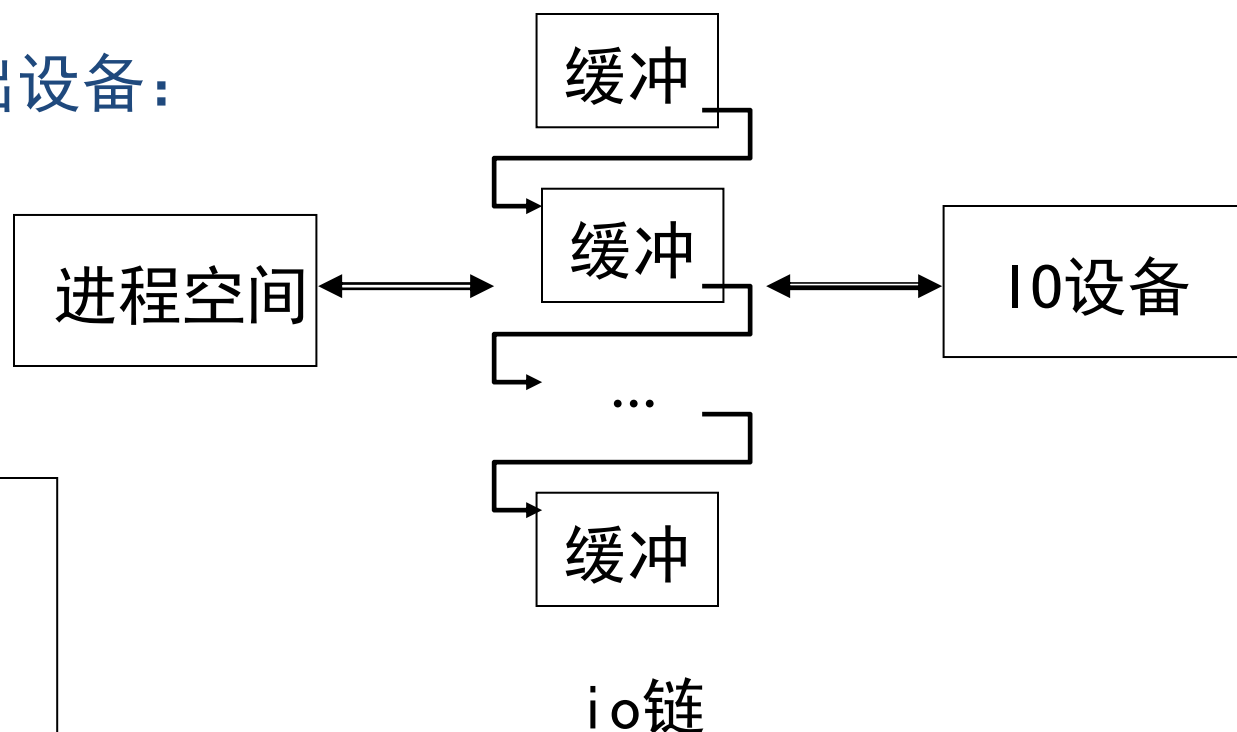
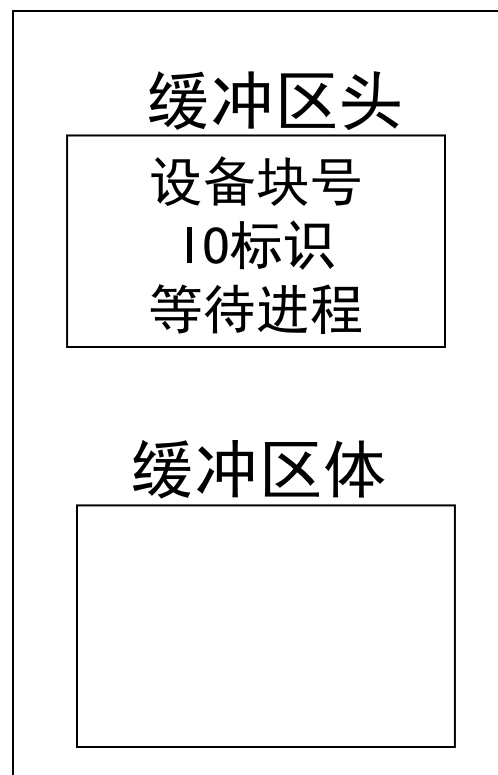
中断方面：



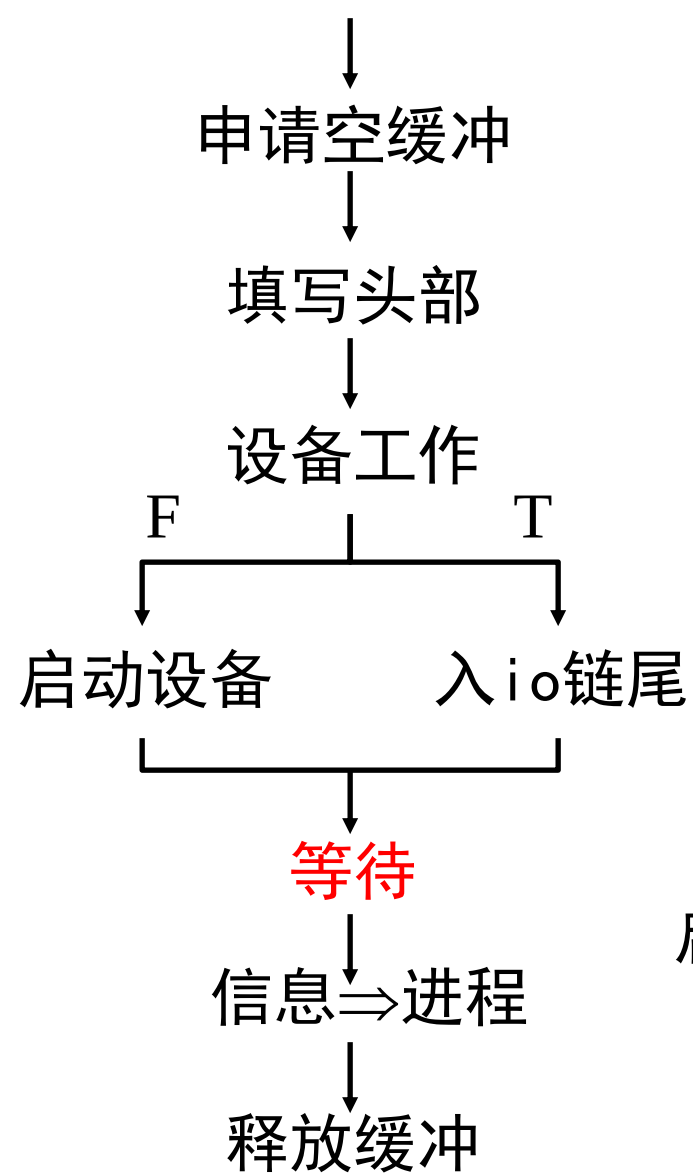
9.7.6 输入输出设备：

(磁带、磁盘)

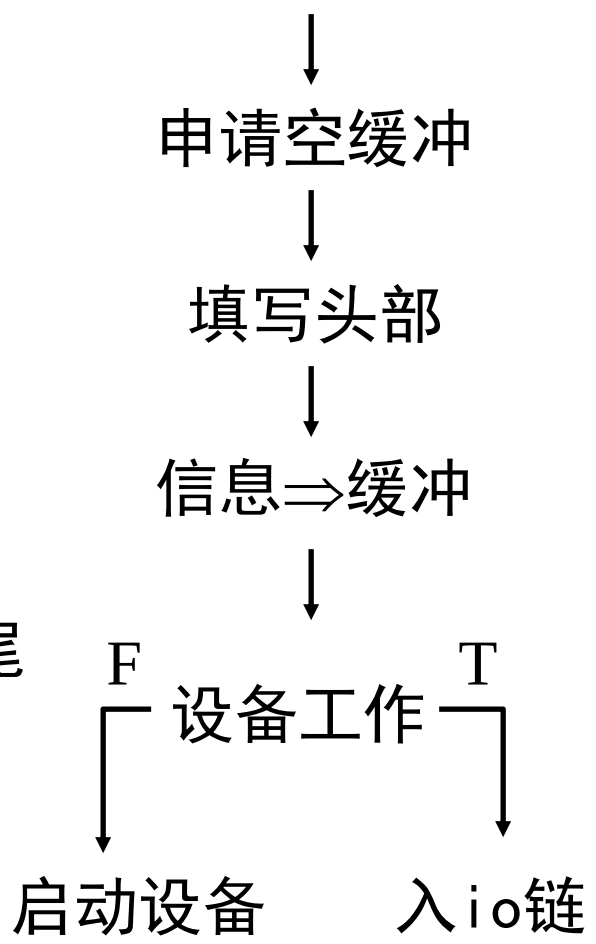
块型缓冲区：



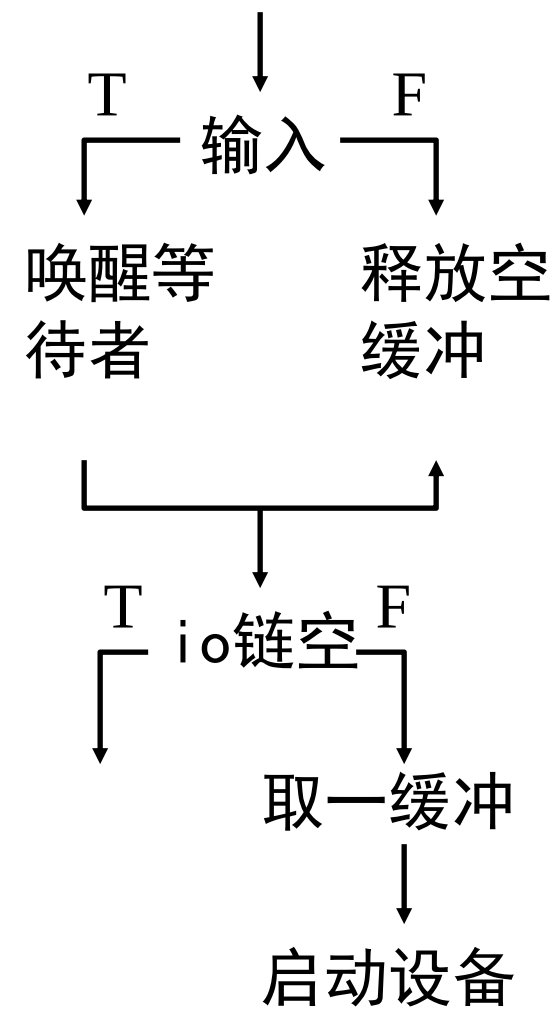
进程方面(输入):



进程方面(输出):



中断方面:



输入每次必等 (无缓存情况下)

UNIX缓冲

- UNIX缓冲目的
 - 缓解处理器和设备速度不匹配的问题
- UNIX缓冲的特点
 - 缓冲区即可以作为缓冲区使用，也可以作为外存中的信息在内存中的高速副本（高速缓存）
- 缓冲和缓存
 - 缓冲：解决速度不匹配的问题
 - 高速缓存：为低速设备中的数据提供高速副本

UNIX缓冲 (Cont.)

- 字符型缓冲
 - 100个缓冲区，长度8字节(6字符+2字节指针)
 - 组成公共缓冲池，所有字符型设备公用
 - 缓冲区或属于`cfreelist`(字符型空闲缓冲区链表)，或属于某字符设备的`io`链(eg. `tty`, `lp`)
- 块型缓冲
 - 50个缓冲区，长度514字节(521字符+2字节指针)
 - 组成公共缓冲池，所有块型设备公用
 - 缓冲区可属于`bfreelist`(块型空闲缓冲区链表) and/or `devtab`

//字符型设备缓冲

```
struct cblock {  
    struct cblock *c_next; //指针  
    char info[6];  
}
```

```
struct cblock *cfreelist; //所有字符型设备的空闲缓冲区
```

//字符型设备的控制结构

```
struct clist          //associated with a character device  
{  
    int c_cc;          //字符计数器，即有多少个字符要输入输出  
    int c_cf;          //指向该设备申请到的缓冲区链头  
    int c_cl;          //指向该设备申请到的缓冲区链尾  
}
```

```
struct buf //块型设备缓冲区头，实现对缓冲区的管理
{
    int      b_flag;           //BUSY, ASYNC, DELWRI, DONE...
    struct   buf *b_forw;      //设备 b 链向前指针
    struct   buf *b_back;      //设备 b 链向后指针
    struct   buf *av_forw;     //设备 av 向前指针
    struct   buf *av_back;     //设备 av 向后指针
    int      b_dev;            //设备名
    int      b_wcount;         //传输计数器
    char      *b_addr;         //缓冲区体，在内存中的低位地址
    char      *b_xmem;         //缓冲区体，在内存中的高位地址
    char      *b_blkno         //缓冲区对应的设备（磁盘块）号
    char      b_error;         //IO传输错误标志
    char      *b_resid;        //传输剩余字节
}buf[NBUF]

char  buffers[NBUF][512]; //块型缓冲区

struct buf bfreelist; //空闲缓冲区头部的链头
```



- 0: 读写标志, 0为读, 1为写, 即输入还是输出
- 1: I/O传输是否完成, 值为1表示缓冲区内容已经写到外存
- 2: I/O传输错误标志
- 3: 占用标志, 缓冲区是否被其它进程所使用, 需互斥
- 6: 是否有进程在等待标志
- 7: 是在开发 UNIX 系统时遗留下来的, 现在已经不再使用
- 8: 异步写标志, 即不需要等待写完返回
- 9: 延迟写标志, 为1为延迟写

```
struct devtab //设备表（每个设备一个），设备控制结构
{
    char    d_active;           //busy flag
    char    d_errcnt;          //error count
    struct  buf *b_forw;        //first buffer for this dev
    struct  buf *b_back;        //last buffer for this dev
    struct  buf *d_actf;        //head of IO queue
    struct  buf *d_actl;        //tail of IO queue
}
```

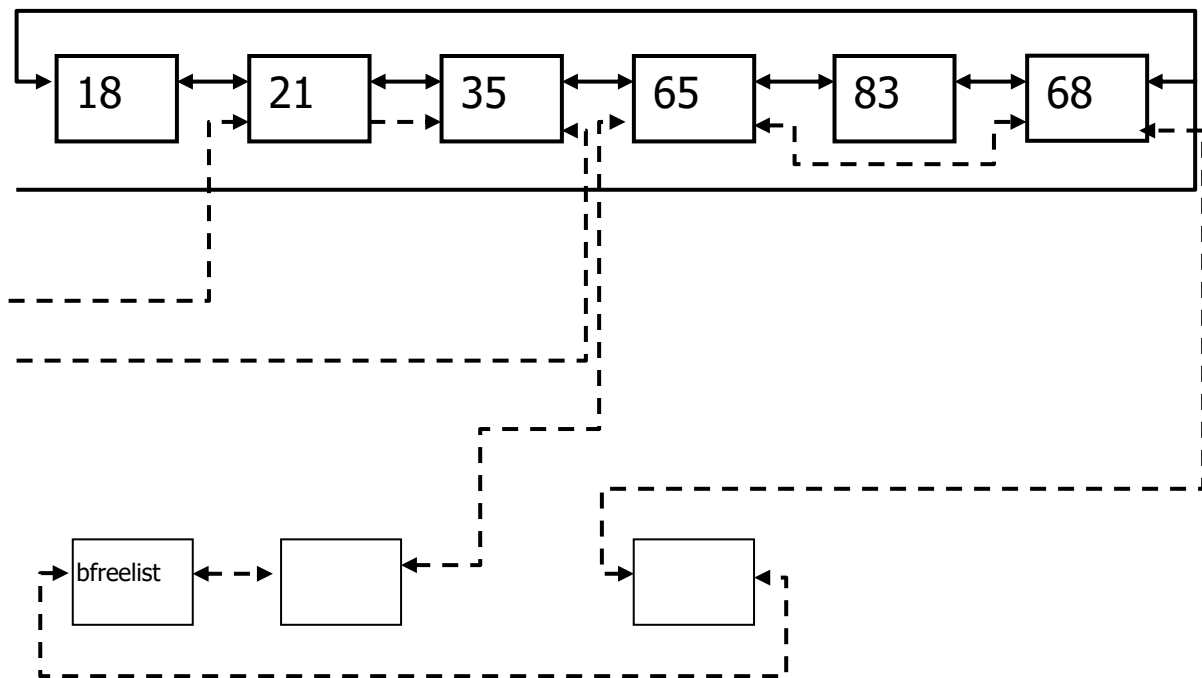
- devtab中包含两个链
 - b链(高速缓存)：链入b链的缓冲区与设备对应块上的内容保持对应关系，对某一磁盘块发出的访问请求，系统先在b链上寻找，只有在找不到时才为其分配一个空闲缓冲块。
 - d链(缓冲)：真正的输入输出队列，为单向链，使用av_forw指针。传送完的缓冲区内容与磁盘上对应块上的内容保持一致。
- 一个块型缓冲区可以同时属于bfreelist链和某一设备的b链
 - 目的是通过预先读和延迟写来增强缓冲效果，提高缓冲区的利用率
 - 当在bfreelist链上的缓冲区被分配其它作用时，才将其由b链摘除
 - 一个缓冲区不会同时属于d链和bfreelist链

```

struct devtab{
    char d_active;
    char d_errcnt;

    struct buf *b_forw;
    struct buf *b_back;

    struct buf *d_actf;
    struct buf *d_actl;
}
    
```



相关操作（系统内部函数，块型设备IO控制的核心函数，非系统调用）：

`getblk(dev,blkno)` //assign a buffer for the given block

`bread(dev,blkno)` //read a block(if necessary), return buf pointer

`breada(dev,blkno,rablkno)` //read in first block, like read;

but also start io on second block

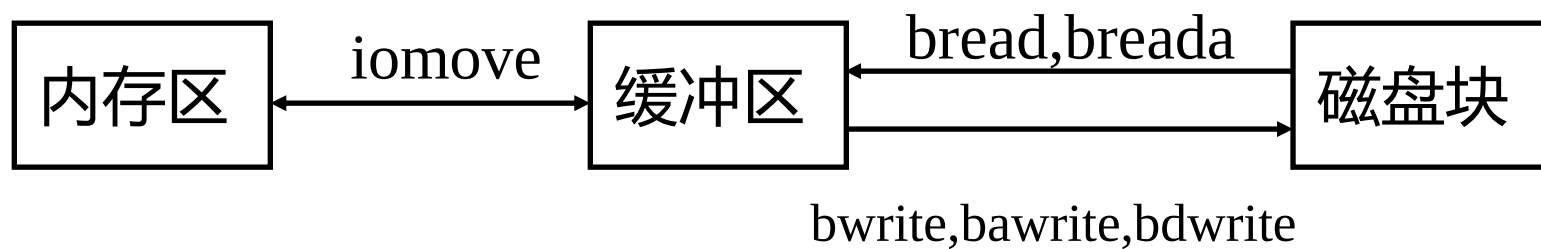
`bwrite(bp)` //write the buffer, wait for completion, then release

`bawrite(bp)` //start the io, release buffer, no wait for completion

`bdwrite(bp)` //release buffer, mark it so that if it is grabbed for
another purpose, it will be written out before
being given up

`brelease(bp)` //release the buffer, with no io implied

块缓冲数据读写



getblk(dev, blkno)

- 功能：为磁盘块分配一个缓冲区
- 参数： dev:设备号, blkno: 设备块号
- 返回：缓冲区（头部）指针bp
- 步骤：
 - 块在b链中，且该块当前空闲（未被其他进程使用）
 - 由av链摘除，标记BUSY，返回缓冲块指针
 - 块在b链中，但BUSY（其它进程在用）
 - 标记B_WANTED, sleep(空闲事件发生)，返回缓冲块指针
 - 块在io链中（该块已经在io链中）
 - 等待io操作结束，返回缓冲块指针
 - 不在b的链中，在av链上取到延迟写的块
 - 写出该块，分配下一个缓冲区
 - 不在b的链中，av链已空
 - 等待任意缓冲区变空闲的事件，分配该缓冲
 - 不在b的链中，在av链上得到空缓冲
 - 由av链摘除，出旧b链，入新b链，填写头部，清B_DONE标志，
 - 返回缓冲块指针

brelease(bp)

- 功能：释放bp所指的缓冲区
- 参数：bp：缓冲区头指针
- 返回：无
- 步骤：
 - If 有等待者 (`b_flag & B_WANTED != 0`), 唤醒;
 - Else
 - bp入av链
 - bfreelist上有等待者, 唤醒

bread(dev, blkno)

- 功能：读设备dev中的块blkno
- 参数：dev:设备号, blkno: 设备块号
- 返回：载有信息的缓冲区bp
- 步骤：
 - bp=getblk(dev, blkno)
 - if (B_DONE) //缓冲区数据有效
 - return(bp) //在cache中得到
 - 填写头部;
 - 启动磁盘读(d_actf/d_actl链)
 - sleep(等待读盘完成事件)
 - return(bp)
- 中断
 - 标记B_DONE
 - 唤醒

breada(dev, blkno, rablkno)

- 参数： dev:设备号, blkno:读块号, rablkno:预读块号
- 返回： blkno缓冲块头部指针rbp
- 步骤：
 - rbp=getblk(dev, blkno)
 - if(!B_DONE) //信息无效
 - 填写头部；启动设备读入(d_actf/d_actl链)
 - rabp=getblk(dev, rablkno)
 - if (B_DONE) //缓冲区从b链得到
 - brelse(rabp) //入av链
 - else //缓冲区从av链得到
 - 填写头部；启动设备读入(d_actf/d_actl链) //中断时入av链
 - iowait(rbp), 标记B_DONE
 - return(rbp)

bwrite, bawrite, bdwrite使用

- 先调用bp=getblk(dev, blkno)指派缓冲；
- 填写缓冲头部；
- 向缓冲区中写入数据，可能整块，或部分；
- 调用：
 - bwrite（同步写出，即等I/O传输完再返回）；
 - bawrite（异步写出，不需要等I/O传输完再返回）；
 - bdwrite（延迟写出）。

bwrite(bp)

- 参数：bp：缓冲区指针
- 步骤：
 - 标记B_DONE
 - 入设备d_act队列(若设备不忙启动设备)
 - if(! B_ASYNC)
 - sleep(等待IO完成事件)
 - 中断
 - 如为同步写，唤醒等待者
 - 如有等待该块(B_WANTED=1)者，唤醒；
 - 否则brelse(bp)，(bp入av链)

bwrite(bp)

- 参数：bp：缓冲区头指针
- 返回：无
- 步骤：
 - `bp->b_flag |= B_ASYNC`
 - `bwrite(bp)`
- 中断
 - 入av队列

bdwrite(bp)

- 参数: bp: 缓冲区指针
- 返回: 无
- 步骤:
 - 标记 `b_flags = B_DELWRI | B_DONE`
 - `brelse(bp)`, (bp入av链)

块型设备管理策略

■ 预先读

- 将即将使用的块读入内存缓存区。
- 预先读是基于文件顺序访问这一假设，系统将其预先读入内存缓冲区，当进程真正访问该块时便可从缓冲区中直接得到。
- `breada(dev, blkno, rblkno)`，其中`blkno`为本次读取的盘块号，`rblkno`为预先读取的盘块号，其所申请到的缓冲区链入`b`链。数据传输完成后中断处理程序将其链入`bfreelist`链，在实际需要此块时可以在`b`链上直接得到，而不再需要执行输入输出操作。
- 由于预测并非完全准确，预先读入块所占用的缓冲区也同时链入`bfreelist`链，当缓冲区资源紧张时刻被分配其它用途。

块型设备管理策略（Cont.）

■ 延迟写

- 当一个缓冲块尚未写满时，暂不将其写入磁盘，这样，以后继续写时，也不必将其再读入内存。
- 另一个好处是当读者欲访问延迟写的块时，可以在缓冲区找到，因而无需执行IO操作。
- `bdwrite(bp)`，参数`bp`为缓冲区头指针，其中包含所有输入输出参数，如`dev`，`blkno`等。该函数只将所对应的缓冲区链入对应设备的`b`链和`bfreelist`链，若以后还访问该块，可以直接在`b`链中得到，该缓冲区中内容比磁盘块上内容新。
- 由于`bfreelist`链中缓冲区是按环形组织，当一个标记为延迟写的缓冲区在`bfreelist`链上被顺序取到，该缓冲区将被链入对应设备的`d`链，从而被写到磁盘，而本次空缓冲区的请求将由`bfreelist`链上的下一个缓冲区来满足。

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.11 输入输出进程

- 专门负责IO传输的进程
 - 另外一种IO模式—服务模式
 - C/S Model
- 特点
 - 界面清晰, 方便使用
 - 两次进程切换, 速度问题
- 工作方式
 - 进程向IO进程发送消息, 由IO进程完成IO
 - 当有多个进程提出IO请求, 则均向IO进程发送消息, IO进程可以按接受消息的次序或某种调度次序完成

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.12 RAID技术

- RAID:
 - Redundant Array of Inexpensive Disks (廉价磁盘冗余阵列)
 - compared with SLEDs (Single Large Expensive Disks)
 - Redundant Array of Independent Disks (独立磁盘冗余阵列)
- Proposed by researchers at UC Berkeley
 - David A. Patterson
- Background
 - disk access speed increases slowly compared with CPU
 - solution: multiple parallel component
- Objective
 - increased disk capacity
 - enhanced performance
 - high reliability

RAID技术

- RAID
 - RAID is a set of physical disks viewed by the operating system as a single logical drive
 - Data are distributed across an array of physical drives
 - Redundant disk capacity is used to store parity information, which guarantees data recoverability in case of disk failure
- Hardware RAID vs. Software RAID
 - hardware based: special controller
 - Windows, UNIX support software RAID
- SCSI RAID vs. IDE RAID (接口类型)
 - performance: SCSI outperforms IDE
 - price: IDE beats SCSI

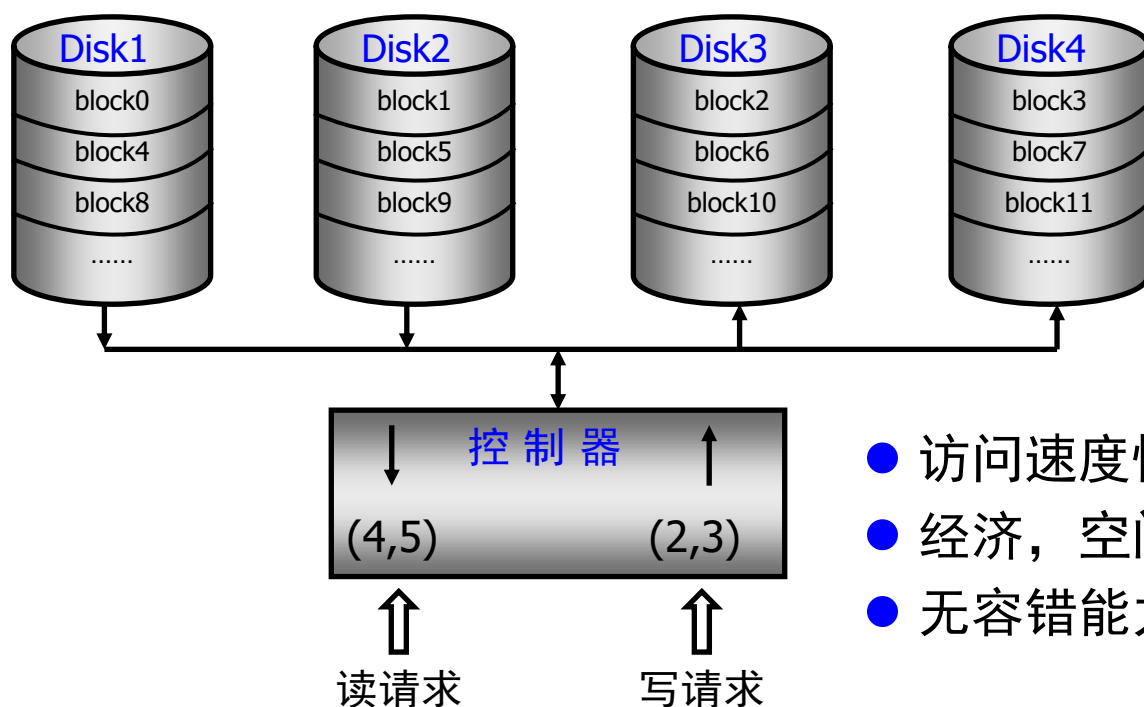
How to organize data
across multiple disks?
Industry standards
scheme

9.12.1 RAID级别

- RAID级别：行业标准规定的数据在多个磁盘上的存放方法。
 - 常见RAID
 - 级别：level0, ..., level5;
- RAID分条(striping)数据存储方式
 - 位级分条(bit-level striping)
 - 块级分条(block-level striping)
- RAID衡量指标
 - 速度：是否支持多个访问同时进行;
 - 可靠性：是否能够发现和改正错误;

Level 0 (数据分条)

- 数据分条以块为单位，连续的数据条（块）循环存放在多个磁盘上。

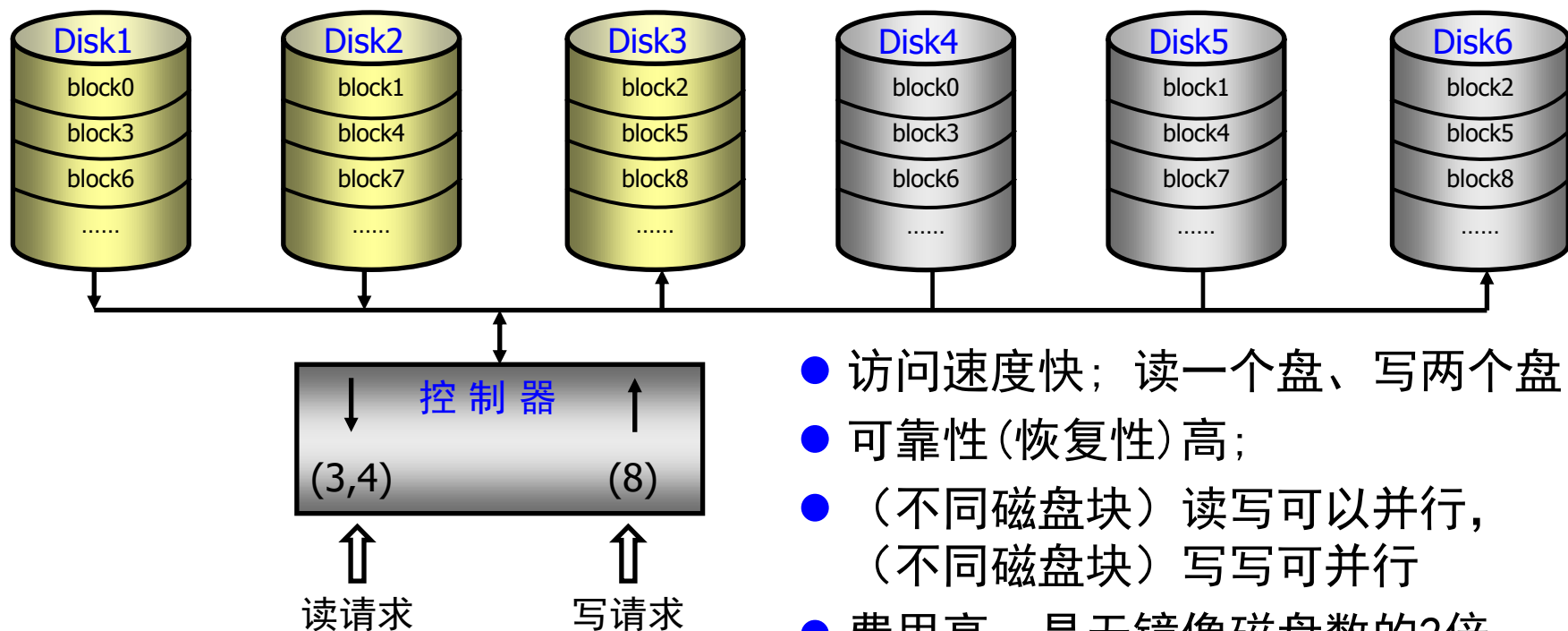


- 访问速度快;
- 经济，空间利用率100%;
- 无容错能力，可靠性差。

例：RAID控制器将请求分解为4条命令，各对应一个磁盘驱动器，使其并行工作（读写不同磁盘时）

Level 1 (镜像, mirroring):

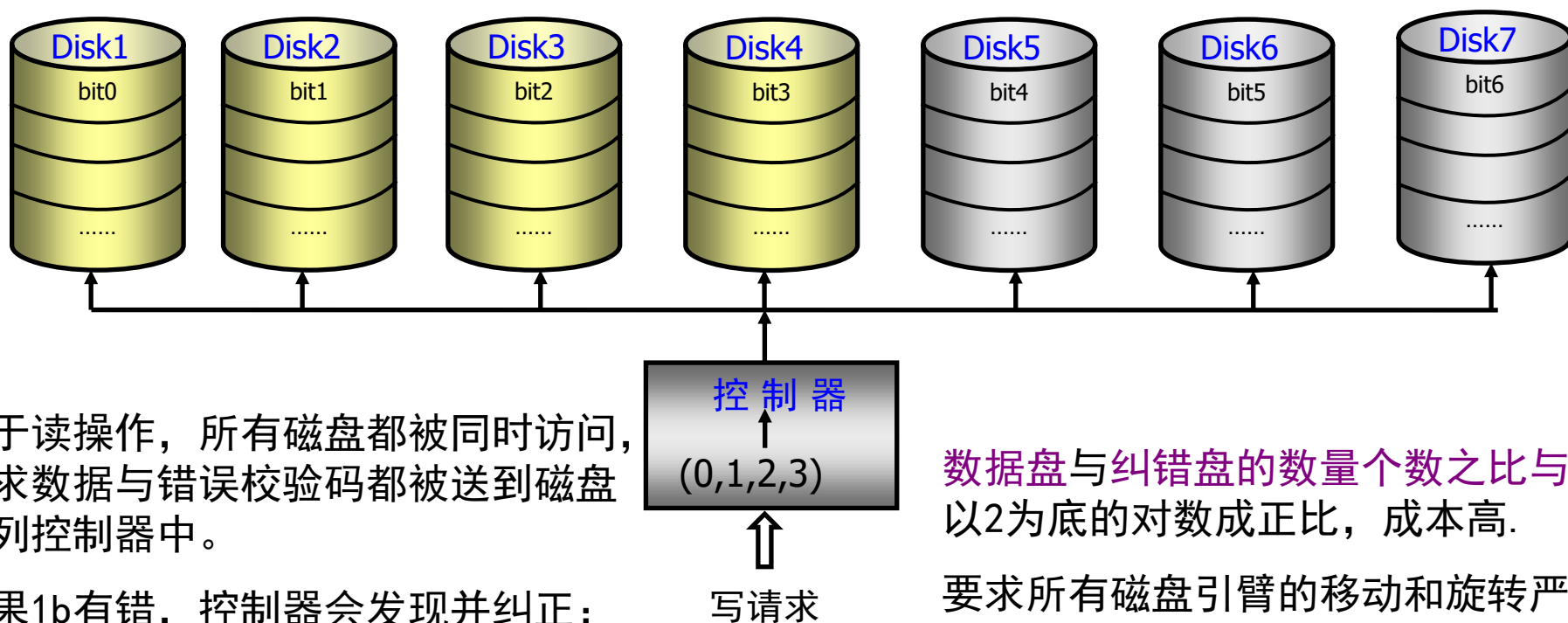
- 数据分条以块为单位，采用分布镜像方式存储，即完全相同的数据重复存放在两个盘上。



- 访问速度快；读一个盘、写两个盘；
- 可靠性(恢复性)高；
- (不同磁盘块) 读写可以并行，
(不同磁盘块) 写写可并行
- 费用高，是无镜像磁盘数的2倍，
空间利用率50%。

Level 2 (位级汉明纠错码校验与恢复)

- 数据以位(bit)为单位分条, 分布存放在多个数据磁盘上, **汉明纠错码存放在纠错磁盘上。**



对于读操作, 所有磁盘都被同时访问, 请求数据与错误校验码都被送到磁盘阵列控制器中。

如果1b有错, 控制器会发现并纠正;

如果2b有错, 可以发现但不能纠正;

对于写操作, 必须同时访问所有数据盘和校验盘。

数据盘与纠错盘的数量个数之比与以2为底的对数成正比, 成本高。

要求所有磁盘引臂的移动和旋转严格同步;

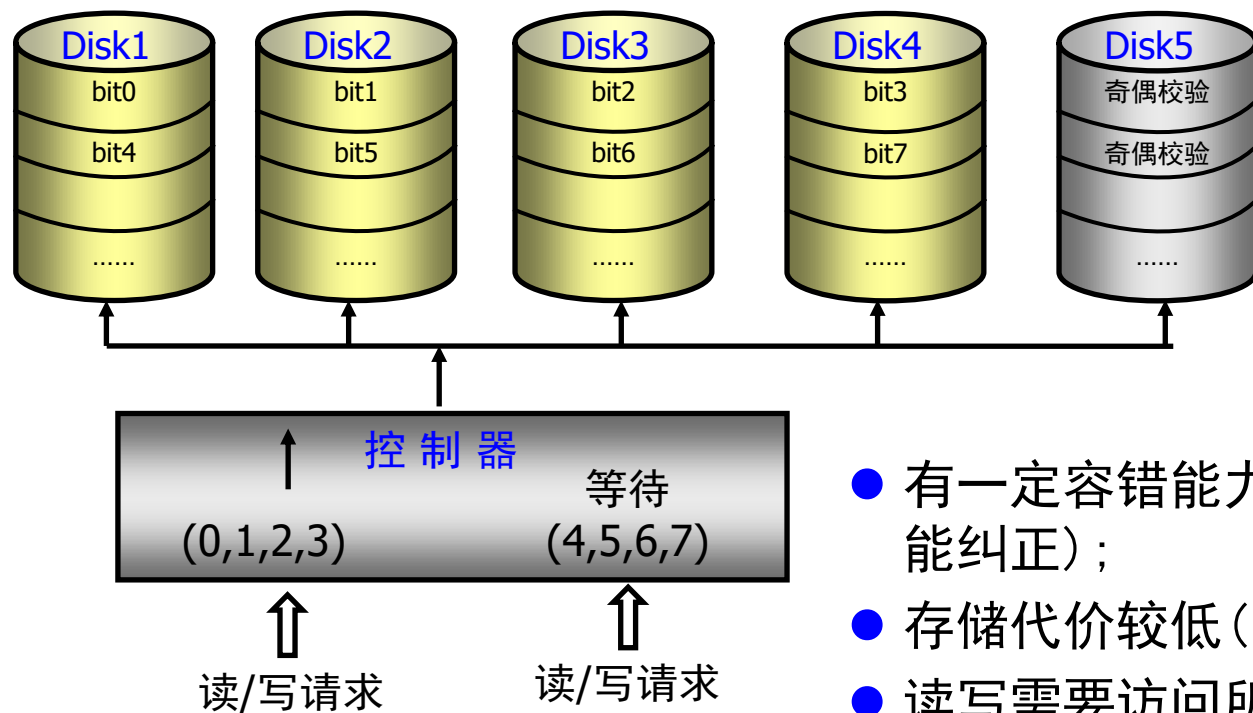
不能同时为多个请求服务, 速度慢

Level 2（位级汉明纠错码校验与恢复）

- 纠错能力强，可靠性高；
 - 发现2个错误，纠正1个错误。
- 需要较多纠错盘存放汉明纠错码，成本较高；
- 不能同时为多个请求服务，速度较慢：
 - 读操作：所有磁盘同时访问，数据与错误校验码被送到磁盘阵列控制器；
 - 写操作：必须同时访问所有数据盘和纠错盘。

Level3 (位级单个奇偶校验)

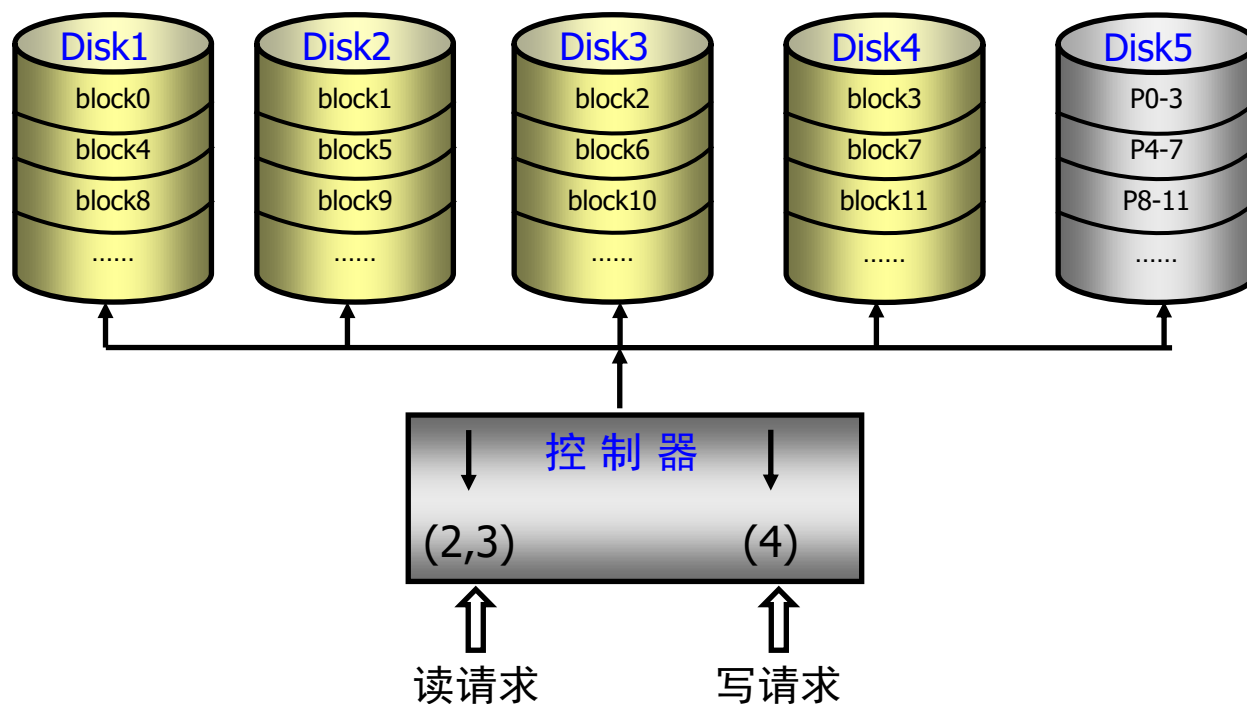
- 数据以位(bit)为单位分条, 分布存放在多个数据磁盘上; 只用一个冗余磁盘存放奇偶校验位。



- 有一定容错能力(发现 1 个错误, 不能纠正);
- 存储代价较低(1个冗余磁盘);
- 读写需要访问所有盘, 多个读写不能并行。

Level 4 (块级异或校验)

- 数据分条以块为单位，用异或运算产生校验信息，
- 校验信息保存在单独的磁盘上。

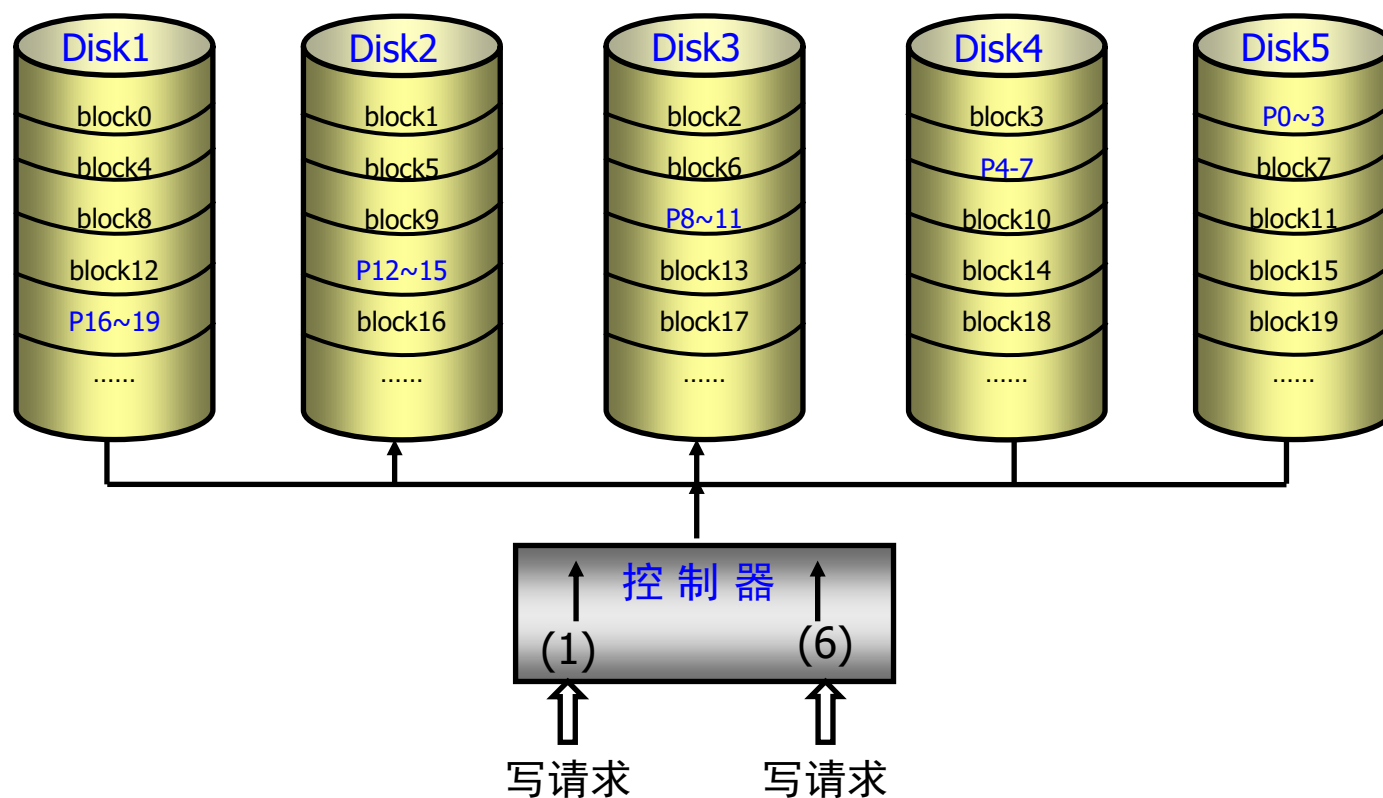


Level 4 (Cont.)

- 读操作不进行异或校验(不需访问校验盘), 可以并行;
- 写操作要更新异或校验信息, 都访问校验盘, 不能并行;
写操作时校验信息更新:
- $P'_{4\sim7} = (\text{block4} \text{ xor } \text{block4}') \text{ xor } p_{4\sim7}$
- 异或校验信息用于磁盘发生故障时数据块的恢复。
- 例如: 若block7所在的Disk4发生故障, 要恢复block7。
- $\text{block7} = p_{4\sim7} \text{ XOR } (\text{block4} \text{ XOR } \text{block5} \text{ XOR } \text{block6})$

Level5 (块级分布式异或校验)

- 数据分条以块为单位，异或校验信息分散循环保存在各磁盘上。



Level 5（块级分布式异或校验）

- 校验码计算：
 - $p4\sim7 = \text{block4} \text{ XOR } \text{block5} \text{ XOR } \text{block6} \text{ XOR } \text{block7}$
- 若block7发生故障，可通过下式恢复
 - $\text{block7} = p4\sim7 \text{ XOR } \text{block4} \text{ XOR } \text{block5} \text{ XOR } \text{block6}$

Level 5 (Cont.)

- 磁盘数量至少为3个;
- 读操作可并行;
- 不涉及相同数据盘和校验盘的写操作可以并行;
- 对于单盘容量为 S 、数量为 N 的磁盘阵列,
- 有效存储容量为: $S \times (N-1)$
- 磁盘利用率为: $(N-1) / N$
- 任意磁盘发生故障, 均可根据其它 $N-1$ 个磁盘恢复;

9.12.1 RAID级别 (Cont.)

RAID 级别的比较

Level	分条粒度	读并行性	写并行性	冗余(容错/开销)
0	块	支持	支持	无/无
1	块	支持	支持	镜像, 强/大
2	位	不支持	不支持	汉明码, 强/大
3	位	不支持	不支持	单个奇偶校验, 较强/较小
4	块	支持	不支持	块级异或校验, 较强/较小
5	块	支持	支持	块级分布异或, 较强/较小

9.12.2 Remark on Software RAID

- Advantages:
 - cost less //成本低，不需要多余的控制器
 - simplicity
- Disadvantages:
 - Performance //速度和可靠性不及硬件RAID
 - boot volume limitation //导引卷不能是RAID
 - level support: only 0, 1, 5 are supported
 - OS compatibility: multiple problem
 - other software compatibility
 - reliability
 - software bugs

Benefit from RAID

- **Increased read/write speed**
- **Improved data reliability**
- **Enlarged storage capacity**
 - **multiple disks viewed as a single volume**

提 纲

9.1设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.13 虚拟设备

■ 概念

- 在一类物理设备上模拟另一类物理设备的技术
 - 利用共享型设备实现的数量较多、速度较快的独占型设备，将独占型设备转化为共享设备
 - 即利用**软件方法**，比如SPooling系统，将一台独占的物理设备虚拟为若干逻辑设备，供若干个用户/进程同时使用

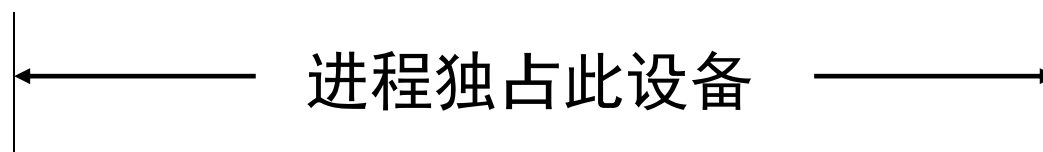
■ 引入

- 用户直接使用独占型设备效率低

虚拟设备引入

■ 用户使用独占型设备活动：

- 申请，使用，使用，…，使用，释放



■ 缺点：

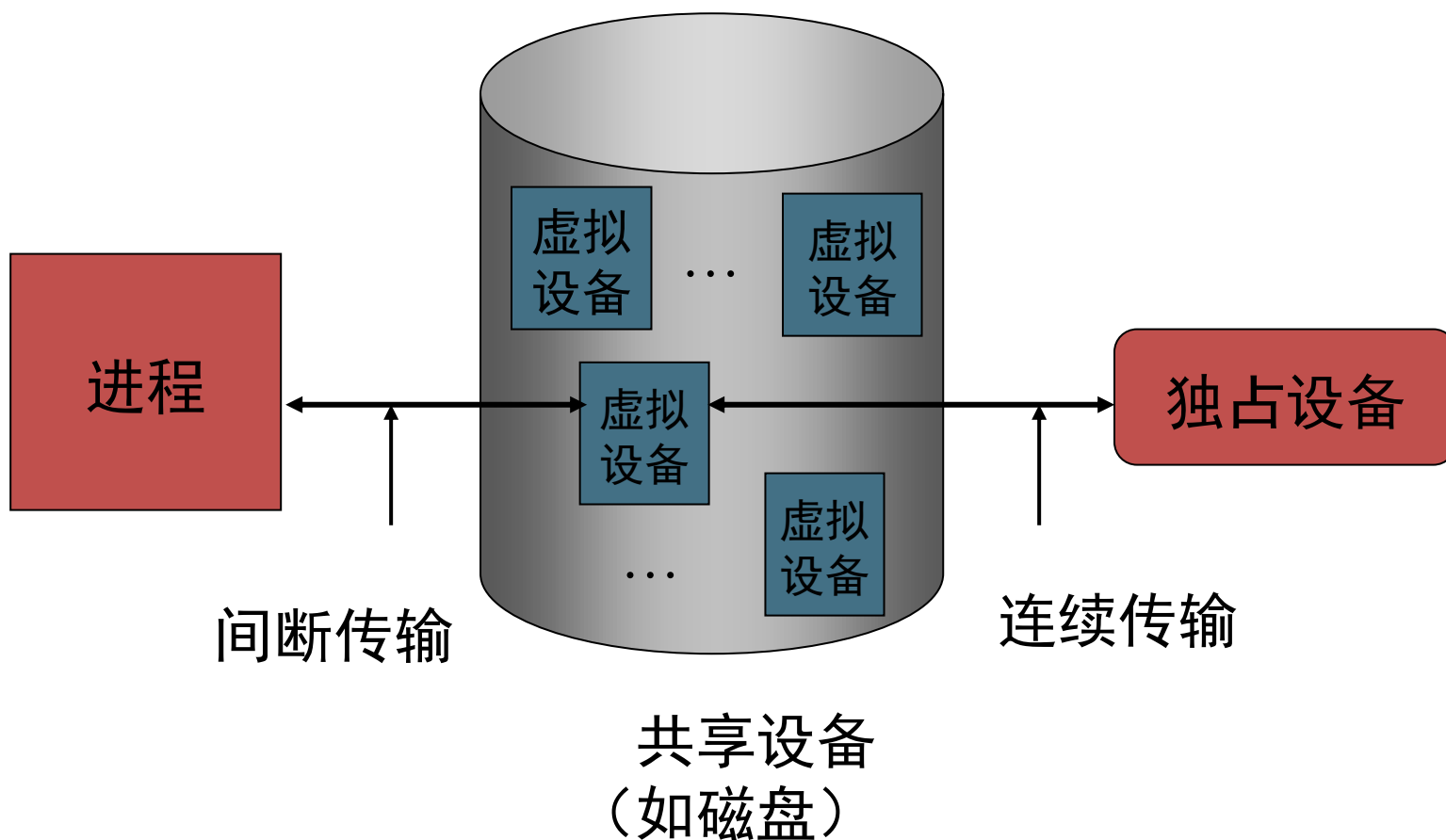
- 速度：CPU与设备速度不匹配
- 设备利用率：占有期间不一定一直使用

■ 实现

- 输入型虚拟设备
- 输出型虚拟设备

■ 方法：

- 在进程与独占型设备之间增加共享设备缓冲



虚拟设备的实现

■ 输入型虚拟设备的实现

- 申请：

- 分配一虚设备
- 分配一实设备
- 信息由实设备⇒虚设备(连续传输)
- 去配该实设备

- 使用：

- 信息由虚设备⇒进程空间(可以间断传输)

- 释放：

- 去配虚设备

虚拟设备的实现

■ 输出型虚拟设备的实现

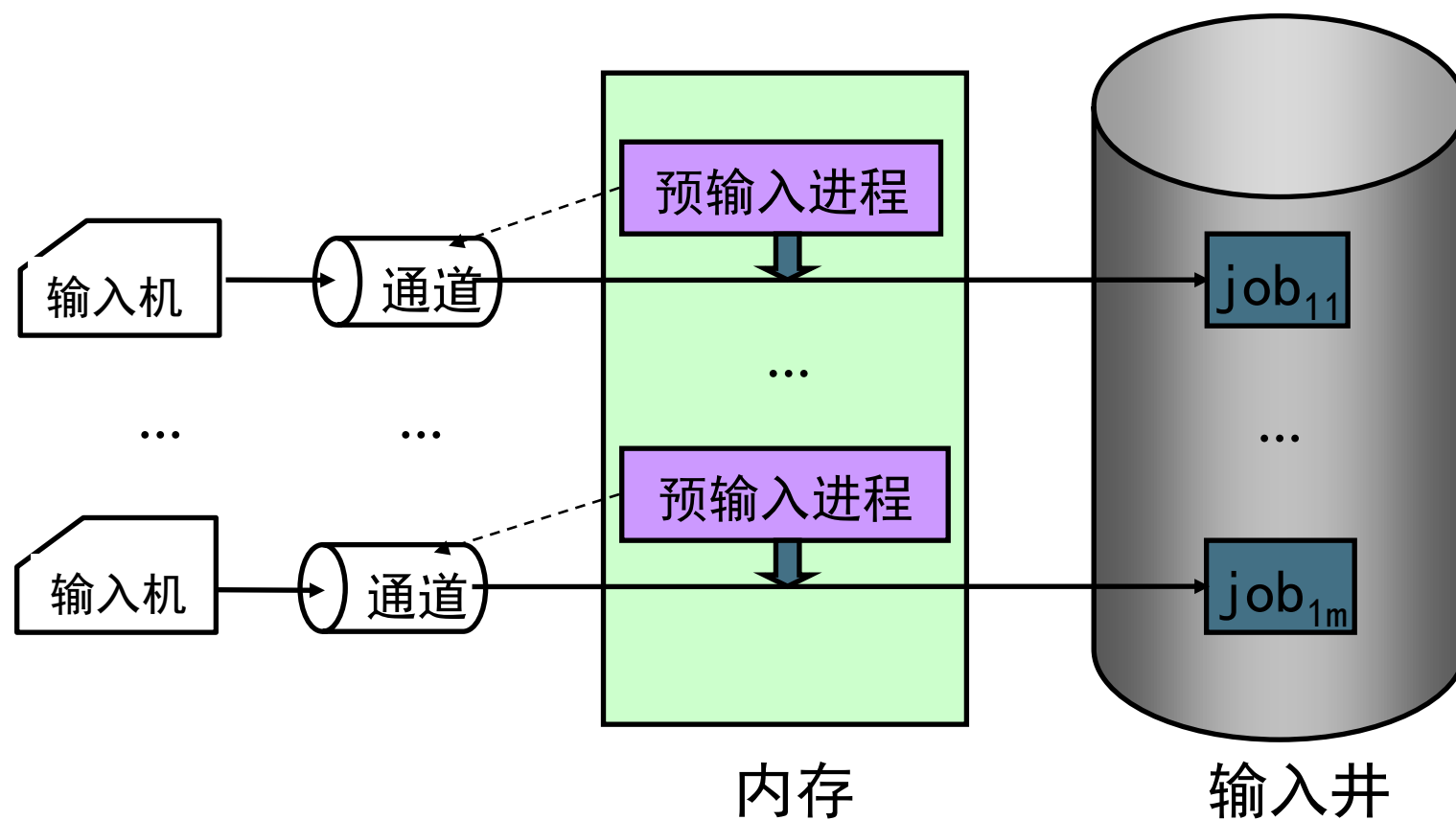
- 申请：
 - 分配一虚设备
- 使用：
 - 信息由进程空间 $\Rightarrow$ 虚设备(间断传输)
- 释放：
 - 分配一实设备
 - 信息由虚设备 $\Rightarrow$ 实设备(连续传输)
 - 去配实设备
 - 去配虚设备

9.13 虚拟设备

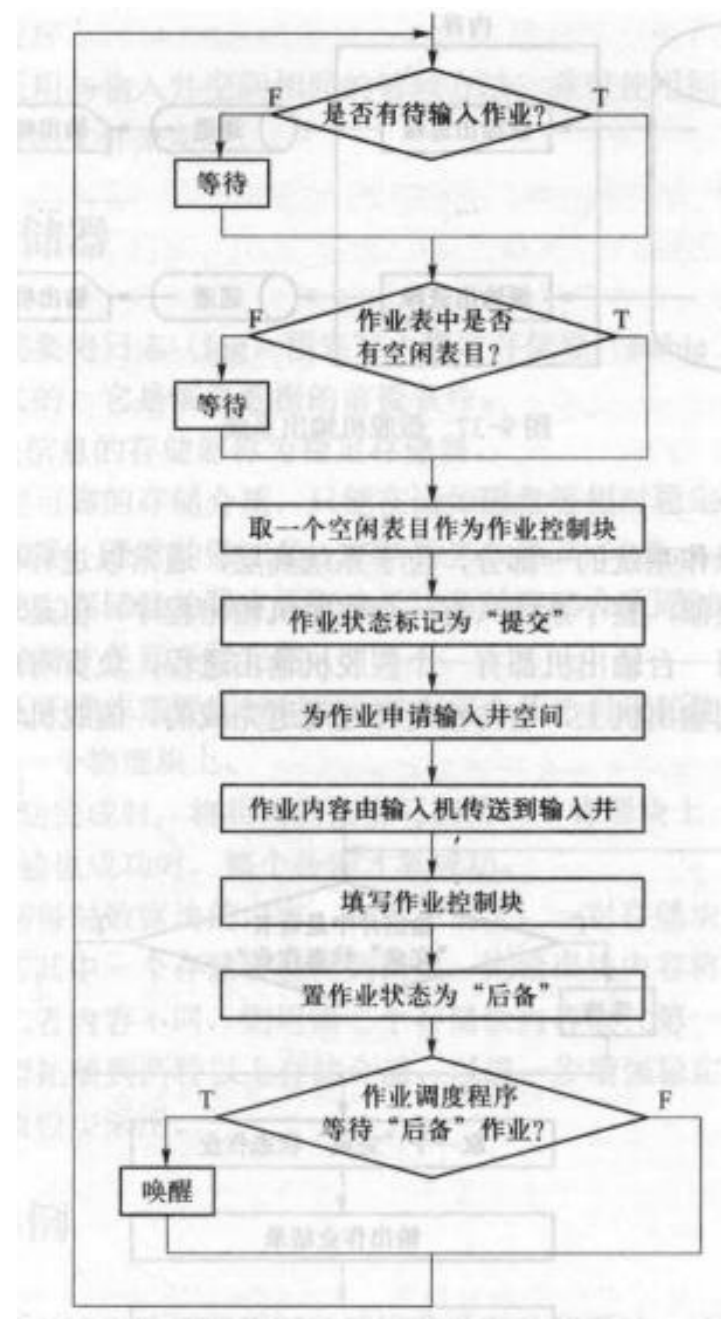
- 虚拟设备的例子- SP00Ling系统结构
 - SP00Ling输入
 - SP00Ling输出

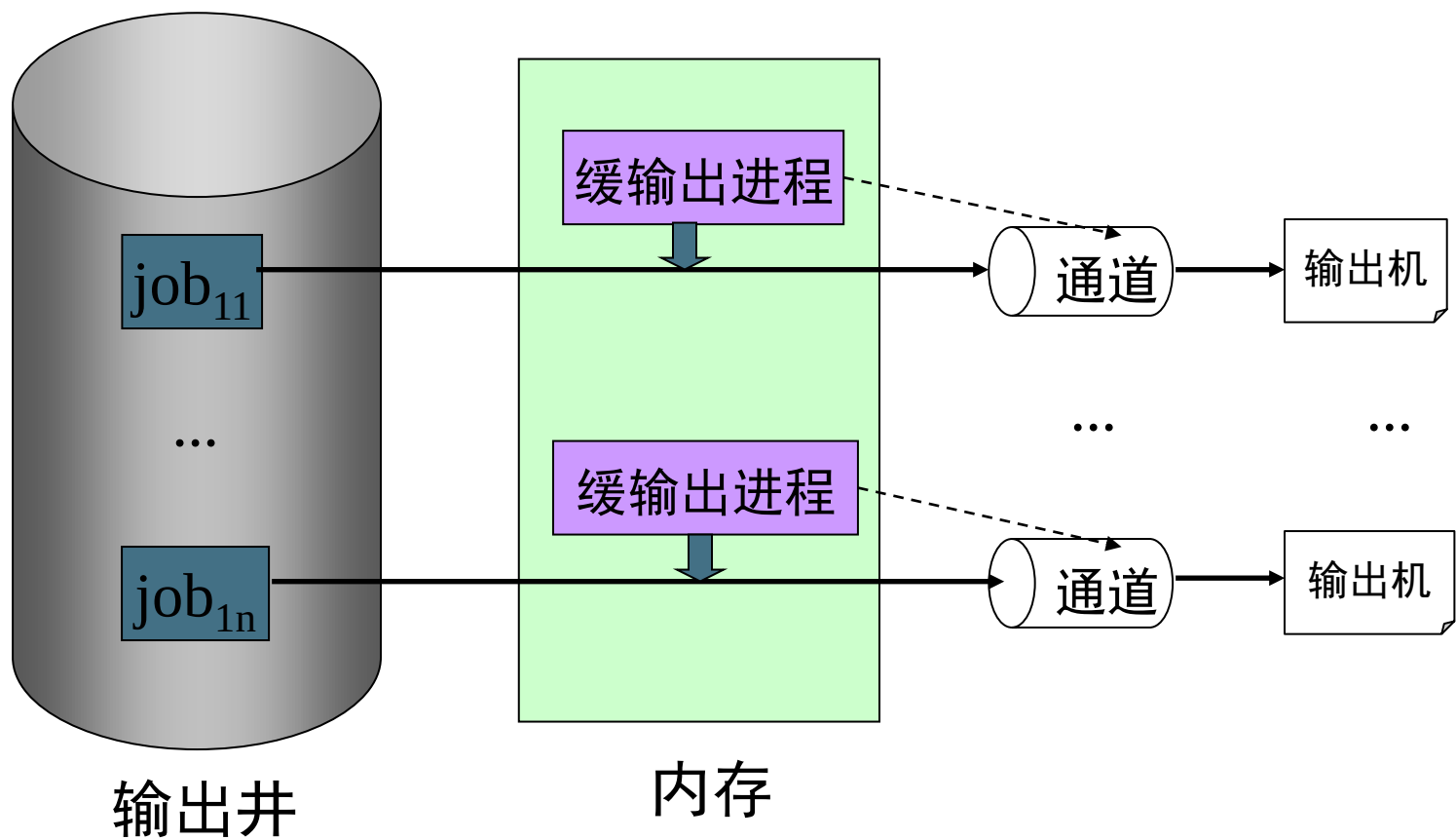
虚拟设备的例子

- SP00Ling
 - SP00Ling输入
 - 作业预输入（输入机⇒输入井）
 - SP00Ling输出
 - 作业缓输出（输出井⇒输出机）

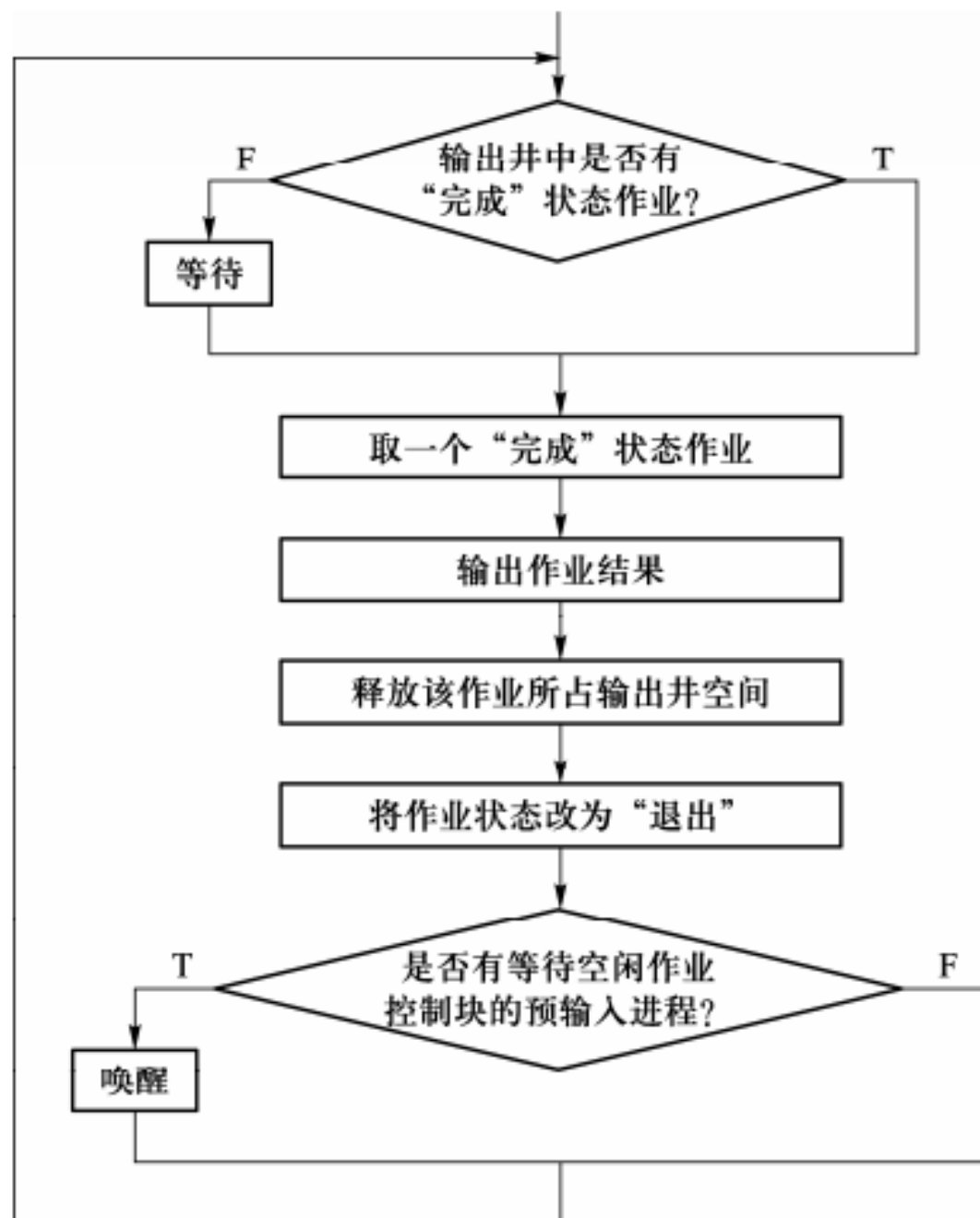


SP00Ling输入程序 (1) vs. SP00Ling输入进程 (n)





SP00Ling输出程序 (1) vs. SP00Ling输出进程 (n)



SP00Ling的结构

- SP00Ling系统的特点
 - 提高了I/O的速度，缓和了CPU与I/O设的速度矛盾；
 - 将独占设备改造为共享设备：利用高速共享设备；
 - 实现了虚拟设备功能：用户都感到独占了一台设备。

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

9.14 稳定存储器

9.15 系统举例-Linux

9.14 稳定存储器 (stable storage)

- 定义

- 不丢失信息的存储器称为稳定存储器

- 实现

- 不存在绝对可靠的存储介质
 - 策略：冗余
 - 在两种失效独立的介质上构建

- 保存

- 将信息写到第一个存储块
 - 上述操作成功后，将相同信息写到第二存储块
 - 仅当第二次传输也成功时，整个保存操作完成

9.14 稳定存储器 (stable storage)

- 实现
 - 恢复
 - 鉴别两个数据块内容
 - 一对信息完全相同，且无检测到错误，正常
 - 其中一块检测到错误，用另一块取代
 - 两块均未检测到错误，但内容不同，用第二块内容取代第一块内容

提 纲

9.1 设备管理的功能和目标

9.2 设备的分类

9.3 设备的物理特性

9.4 数据传输方式

9.5 I/O传输层次

9.6 同步I/O与异步I/O

9.7 设备分配与去配

9.8 设备驱动

9.9 设备调度

9.10 缓冲与缓存

9.11 输入输出进程

9.12 RAID技术

9.13 虚拟设备

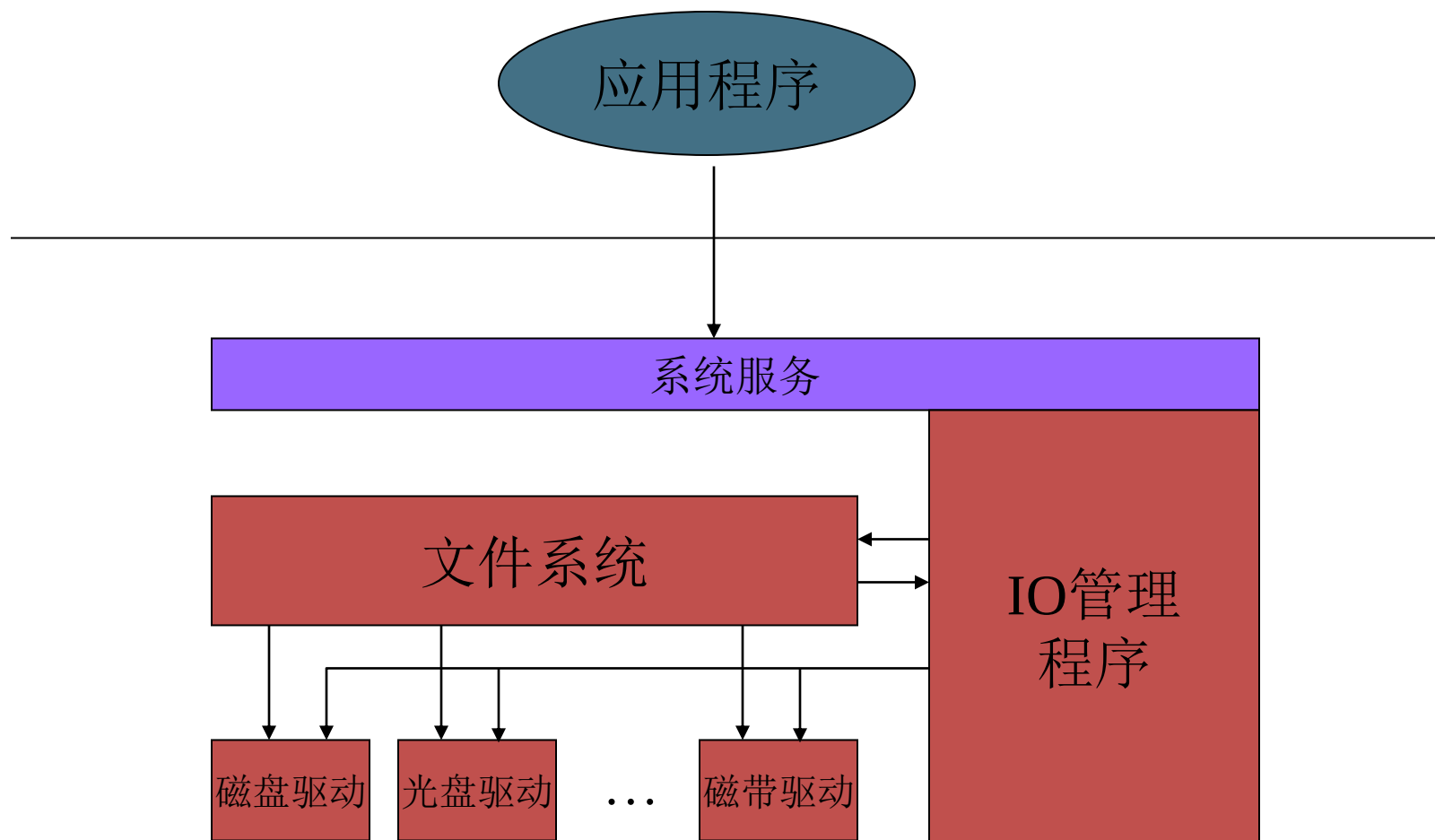
9.14 稳定存储器

9.15 系统举例-Linux

9.15 系统举例—Linux

- 涉及内容
 - 设备驱动，设备缓冲，文件管理
- 主要特点
 - 允许系统管理员动态添加或删除设备，其相应的驱动程序通常以DLL的形式动态加载，设备驱动程序可以采用高级语言编写，移植性好。
 - 除NTFS外，支持多种可装卸的文件系统，如FAT32、OS/2、HPFS、CDFS等。支持映像文件I/O。
 - 设备管理子系统以I/O请求包(I/O Request Package, IRP)方式驱动，每个I/O请求均可以表示为I/O请求包，I/O管理器负责协调所有各组成部分之间的相互作用。
 - 通过系统进程(线程)提供异步I/O，在I/O请求的进程(线程)不需要等待I/O传输结果的情况下，请求者可以立即返回，具体I/O操作由系统线程负责完成。异步I/O是通过异步过程调用(Asynchronous Procedure Call, APC)实现的。

Windows IO子系统



结束

- 设备及其分类
- 设备的物理特性
- I/O 传输方式
- 设备的分配与去配
- 设备驱动
- I/O传输层次
- 设备调度
- 缓冲技术
- UNIX缓冲
- RAID技术
- 虚拟设备